Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 12_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement:

Raju has a hot dog of a certain length that he wishes to divide strategically to maximize the number of friends he can serve. The effectiveness of a given partition is determined by the product of its segment lengths. Raju's goal is to determine the optimal way to cut the hot dog such that this product is maximized, thereby maximizing the number of friends he can serve.

Example

Input:

4

Output:

Maximum friends served 4

vtu29414

Explanation:

For a hot dog of length 4, possible partitions and their corresponding products are:

1, 3 (1 × 3) = 3

2, 2 (2 × 2) = 4 (Maximum)

3, 1 (3 × 1) = 3

1, 1, 2 (1 × 1 × 2) = 2

1, 2, 1 (1 × 2 × 1) = 2

2, 1, 1 (2 × 1 × 1) = 2

1, 1, 1, 1 (1 × 1 × 1 × 1) = 1

Thus, the optimal partition results in a maximum product of 4, serving the highest number of friends.

Input Format

The input consists of a single integer n, representing the length of the hot dog.

Output Format

The output prints the maximum number of friends served.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
Output: Maximum friends served 4

Answer

```
// You are using GCC
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
```

```
    cin >> n;

    if (n <= 3) {
        cout << "Maximum friends served " << (n - 1);
        return 0;
    }

    long long product = 1;

    while (n > 4) {
        product *= 3;
        n -= 3;
    }

    product *= n;

    cout << "Maximum friends served " << product;

    return 0;
}
```

Status : Wrong **Marks : 0/10**

2. Problem Statement

Imagine you are given a set of items, each with a weight, a value, and a penalty value. You need to solve the Fractional Knapsack problem with the objective of maximizing the total value while minimizing penalties. Implement a greedy algorithm to choose items that best meet this objective.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

Maximum total value while minimizing penalties: 14.00

Explanation:

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 1: (7 - 1) / 2 = 3

Item 2: (12 - 4) / 3 = 2

Start filling the knapsack:

Add all of Item 1 (Weight: 2, Value: 7 - 1 = 6)

Add 1 unit of Item 2 (Weight: 3, Value: 12 - 4 = 8)

The total value is 6 + 8 = 14.

So, the correct output for the given input is 14.

Example:

Input:

2

5 20 2

10 30 5

15

Output:

Maximum total value while minimizing penalties: 43.00

Explanation:

Sort the items based on the modified value-to-weight ratios (value - penalty) / weight:

Item 2: (30 - 5) / 10 = 2.5

Item 1: (20 - 2) / 5 = 3.6

Start filling the knapsack:

Add all of Item 1 (Weight: 5, Value: 20 - 2 = 18)

Add 1 unit of Item 2 (Weight: 10, Value: 30 - 5 = 25)

Now, let's calculate the total value of the knapsack:

Total Weight = 5 + 10 = 15 (fits within the knapsack's capacity)

Total Value = 18 + 25 = 43

Input Format

The first line contains an integer 'N' representing the number of items.

The next 'N' lines contain information for each item:

Three integers 'weight,' 'value,' and 'penalty,' are separated by spaces, where 'weight' is the weight of the item, 'value' is the value of the item, and 'penalty' is the penalty for taking the item.

The last line contains an integer 'knapsackWeight' representing the weight limit of the knapsack.

Output Format

The output prints "Maximum total value while minimizing penalties: " followed by the maximum total value achievable while minimizing penalties, rounded to two decimal places.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 2
2 7 1
3 12 4
5

vtu29414

Output: Maximum total value while minimizing penalties: 14.00

Answer

```
// You are using GCC
#include <bits/stdc++.h>
using namespace std;

struct Item {
    double weight, value, penalty;
    double ratio;
};

int main() {
    int n;
    cin >> n;

    vector<Item> items(n);

    for (int i = 0; i < n; i++) {
        cin >> items[i].weight >> items[i].value >> items[i].penalty;
        items[i].ratio = (items[i].value - items[i].penalty) / items[i].weight;
    }

    double capacity;
    cin >> capacity;

    sort(items.begin(), items.end(), [](Item a, Item b) {
        return a.ratio > b.ratio;
    });

    double totalValue = 0.0;

    for (int i = 0; i < n && capacity > 0; i++) {
        if (items[i].weight <= capacity) {
            totalValue += (items[i].value - items[i].penalty);
            capacity -= items[i].weight;
        } else {
            double fraction = capacity / items[i].weight;
            totalValue += fraction * (items[i].value - items[i].penalty);
            capacity = 0;
        }
    }
```

```
    cout << fixed << setprecision(2);
    cout << "Maximum total value while minimizing penalties: " << totalValue;

    return 0;
}
```

Status : Wrong **Marks : 0/10**

3. Problem Statement

There are n participants in a race. Each participant is assigned a performance score given in the integer array performance.

You are responsible for awarding medals to these participants, subject to the following conditions:

Each participant must receive at least one medal.

Participants with a higher performance score than their neighbors must receive more medals than their neighbors.

Return the minimum number of medals you need to distribute to the participants.

Example 1:

Input: ratings = [1,0,2]

Output: 5

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 1 medal (since their performance is the lowest).

Participant 3 gets 2 medals (since their performance is higher than Participant 2).

The total number of medals required is 5.

Example 2:

Input: ratings = [1,2,2]

Output: 4

Explanation:

Participant 1 gets 1 medal.

Participant 2 gets 2 medals (since their performance is better than Participant 1).

Participant 3 gets 1 medal (because their performance is equal to Participant 2, and they don't need more medals).

The total number of medals required is 4.

Input Format

The first line consists of an integer n representing the number of participants.

The second line consists of n space-separated integers representing the performance scores of the participants.

Output Format

An integer representing the minimum number of medals required to satisfy the conditions.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 0 2
Output: 5

Answer

```
// You are using GCC
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
```

```
    cin >> n;

    vector<int> ratings(n);
    for (int i = 0; i < n; i++) {
        cin >> ratings[i];
    }

    vector<int> medals(n, 1);

    // Left to right pass
    for (int i = 1; i < n; i++) {
        if (ratings[i] > ratings[i - 1]) {
            medals[i] = medals[i - 1] + 1;
        }
    }

    // Right to left pass
    for (int i = n - 2; i >= 0; i--) {
        if (ratings[i] > ratings[i + 1]) {
            medals[i] = max(medals[i], medals[i + 1] + 1);
        }
    }

    int total = 0;
    for (int m : medals) {
        total += m;
    }

    cout << total;

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

4. Problem Statement

You are planning a circular road trip that passes through several gas stations. Each station provides a certain amount of fuel, and traveling from one station to the next consumes a specific amount of fuel.

Your goal is to determine a starting gas station index from which you can begin your journey such that you complete the entire circuit in a clockwise direction without running out of fuel at any point.

Given two integer arrays:

gas[i] represents the fuel available at the i-th station,

cost[i] represents the fuel required to travel from station i to station i+1 (circularly),

Return the starting station index that allows completing the full trip.

If no such station exists, return -1.

Example 1

Input:

5

1 2 3 4 5

3 4 5 1 2

Output:

3

Explanation:

Start at station 3 (index 3) and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 4. Your tank = 4 - 1 + 5 = 8

Travel to station 0. Your tank = 8 - 2 + 1 = 7

Travel to station 1. Your tank = 7 - 3 + 2 = 6

Travel to station 2. Your tank = 6 - 4 + 3 = 5

Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.

Therefore, return 3 as the starting index.

vtu29414

Example 2

Input

3

2 3 4

3 4 3

Output:

-1

Explanation:

You can't start at station 0 or 1, as there is not enough gas to travel to the next station.

Let's start at station 2 and fill up with 4 units of gas. Your tank = 0 + 4 = 4

Travel to station 0. Your tank = 4 - 3 + 2 = 3

Travel to station 1. Your tank = 3 - 3 + 3 = 3

You cannot travel back to station 2, as it requires 4 units of gas but you only have 3.

Therefore, you can't travel around the circuit once no matter where you start.

Input Format

The first line of input consists of an integer N, representing the number of gas stations.

The second line of input consists of N space-separated integers representing the amount of gas available at each station.

The third line of input consists of N space-separated integers representing the cost of traveling from each station to the next.

Output Format

The output prints an integer representing the index of the starting gas station that allows you to complete the circuit, or -1 if no such station exists.

vtu29414

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
3 4 5 1 2

Output: 3

Answer

```
// You are using GCC
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> gas(n), cost(n);

    for (int i = 0; i < n; i++) cin >> gas[i];
    for (int i = 0; i < n; i++) cin >> cost[i];

    int totalGas = 0, totalCost = 0;

    for (int i = 0; i < n; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
    }

    if (totalGas < totalCost) {
        cout << -1;
        return 0;
    }

    int start = 0;
    int tank = 0;

    for (int i = 0; i < n; i++) {
        tank += gas[i] - cost[i];
```

```
        if (tank < 0) {
            start = i + 1;
            tank = 0;
        }
    }

    cout << start;

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 12_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Given an array of non-negative integers, the task is to find the minimum number of elements such that their sum should be greater than the sum of the rest of the elements of the array.

Example:

Input: arr[] = [3 , 1 , 7, 1]

Output: 1

Explanation: Smallest subset is {7}. Sum of this subset is greater than the sum of all other elements left after removing subset {7} from the array

Input: arr[] = [2 , 1 , 2]

Output: 2

Explanation: Smallest subset is {2 , 1}. Sum of this subset is greater than the sum of all other elements left after removing subset {2 , 1} from the array

Input Format

The first line contains an Integer n, representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single integer representing the minimum number of elements whose sum is greater than the sum of the remaining elements.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 1 7 1

Output: 1

Answer

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> arr(n);
    int totalSum = 0;

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        totalSum += arr[i];
    }

    sort(arr.begin(), arr.end(), greater<int>());
```

```
    int currSum = 0;
    int count = 0;

    for (int i = 0; i < n; i++) {
        currSum += arr[i];
        count++;

        if (currSum > totalSum - currSum) {
            break;
        }
    }

    cout << count;

    return 0;
}
```

Status : Wrong **Marks : 0/10**

2. Problem Statement

Given an array prices[] of n different candies, where prices[i] represents the price of the ith candy and all prices are distinct. The store has an offer where for every candy you buy, you can get up to k other different candies for free. Find the minimum and maximum amount of money needed to buy all the candies.

Note: In both cases, you must take the maximum number of free candies possible during each purchase.

Examples:

Input: prices[] = [3, 2, 1, 4], k = 2

Output: [3, 7]

Explanation: For minimum cost, buy the candies costing 1 and 2. for maximum cost, buy the candies costing 4 and 3.

Input: prices[] = [3, 2, 1, 4, 5], k = 4

Output: [1, 5]

Explanation: For minimum cost, buy the candy costing 1. for maximum cost, buy the candy costing 5.

Input Format

The first line contains an Integer n, representing the number of candies.

The second line contains n space-separated integers, representing the prices of the candies.

The third line contains an integer k, representing the number of free candies per purchase.

Output Format

The output prints two integers separated by a space:

minCost maxCost

representing the minimum and maximum money needed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
3 2 1 4
2
Output: 3 7

Answer

```
// You are using GCC
#include <stdio.h>
using namespace std;

int main() {
    int n;
    cin >> n;
```

```
    vector<int> prices(n);
    for (int i = 0; i < n; i++) cin >> prices[i];

    int k;
    cin >> k;

    sort(prices.begin(), prices.end());

    // Minimum cost
    int l = 0, r = n - 1;
    long long minCost = 0;

    while (l <= r) {
        minCost += prices[l];
        l++;

        r -= k;
    }

    // Maximum cost
    l = 0, r = n - 1;
    long long maxCost = 0;

    while (l <= r) {
        maxCost += prices[r];
        r--;

        l += k;
    }

    cout << minCost << " " << maxCost;

    return 0;
}
```

Status : Wrong **Marks : 0/10**

3. Problem Statement

Given a fraction represented by two integers — a numerator n and a denominator d — express the fraction as a sum of unique unit fractions

(fractions with numerator 1). This representation is known as the Egyptian Fraction.

Write a program to ensure that the given fraction is converted into its Egyptian Fraction form.

Input Format

The input contains two space-separated integers n and d, representing the numerator and denominator of the fraction.

Output Format

The output prints the Egyptian Fraction representation of the given fraction in the form of unit fractions separated by +.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 14

Output: 1/3 + 1/11 + 1/231

Answer

```
// You are using GCC
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, d;
    cin >> n >> d;

    bool first = true;

    while (n != 0) {
        int x = (d + n - 1) / n; // ceil(d/n)

        if (!first) {
            cout << " + ";
        }
```

```
        cout << "1/" << x;
        first = false;

        // update fraction: n/d - 1/x
        n = n * x - d;
        d = d * x;

        // reduce fraction using gcd
        int g = __gcd(n, d);
        if (g != 0) {
            n /= g;
            d /= g;
        }
    }

    return 0;
}
```

Status : Wrong **Marks : 0/10**

4. Problem Statement

Emily is building a data compression feature in her file organizing system. She wants filenames to have no repeated characters and appear in the smallest possible order so users can locate files easily. For example, if a corrupted filename is "cbacdcbc", she wants it cleaned up as "acdb" — which preserves one instance of each letter, keeps relative order, and is the smallest alphabetically.

This function helps ensure filenames are clean, minimal, and alphabetically organized!

Example 1:

Input: s = "bcabc"

Output: "ABC"

Example 2:

Input: s = "cbacdcbc"

Output: "acdb"

Input Format

The first line of input contains a string num representing a non-negative integer.

The second line contains an integer k representing how many digits to remove.

Output Format

The output prints the string representing the smallest possible number after removing exactly k digits.

Leading zeros should be removed, except if the number is zero itself.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: bcabc
Output: abc

Answer

```
// You are using GCC
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    cin >> s;

    vector<int> freq(26, 0);
    vector<bool> inStack(26, false);
    stack<char> st;

    for (char c : s) {
        freq[c - 'a']++;
    }

    for (char c : s) {
        int idx = c - 'a';
```

```
        freq[idx]--;

        if (inStack[idx]) continue;

        while (!st.empty() && st.top() > c && freq[st.top() - 'a'] > 0) {
            inStack[st.top() - 'a'] = false;
            st.pop();
        }

        st.push(c);
        inStack[idx] = true;
    }

    string result = "";
    while (!st.empty()) {
        result += st.top();
        st.pop();
    }

    reverse(result.begin(), result.end());

    cout << result;

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

5. Problem Statement

Given the arrival and departure times of all trains that reach a railway station, find the minimum number of platforms required for the station so that no train is kept waiting. Consider that all the trains arrive on the same day and leave on the same day.

Arrival and departure times can never be the same for a train, but we can have the arrival time of one train equal to the departure time of the other. At any given instance, the same platform cannot be used for both the departure of a train and the arrival of another train. In such cases, we need different platforms.

Example 1

Input:

N=6

arr[] = {0900, 0940, 0950, 1100, 1500, 1800}, dep[] = {0910, 1200, 1120, 1130, 1900, 2000}

Output: 3

Explanation: A minimum of three platforms are required to safely arrive and depart all trains.

Input Format

The first line consists of an integer N, which represents the total number of trains.

The second line consists of N space-separated integers representing the arrival time.

The third line consists of N space-separated integers representing the departure time.

Output Format

The output displays an integer which is the minimum number of platforms required to safely arrive and depart the train.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
0900 0940 0950 1100 1500 1800
0910 1200 1120 1130 1900 2000

Output: 3

Answer

// You are using GCC

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> arr(n), dep(n);

    for (int i = 0; i < n; i++) cin >> arr[i];
    for (int i = 0; i < n; i++) cin >> dep[i];

    sort(arr.begin(), arr.end());
    sort(dep.begin(), dep.end());

    int i = 0, j = 0;
    int platforms = 0, maxPlatforms = 0;

    while (i < n && j < n) {
        if (arr[i] <= dep[j]) {
            platforms++;
            i++;
        } else {
            platforms--;
            j++;
        }

        maxPlatforms = max(maxPlatforms, platforms);
    }

    cout << maxPlatforms;

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 12_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 10

Section 1 : MCQ

1. The first step in the naive greedy algorithm is _____.

Answer

adding flows with higher values

Status : Wrong ***Marks : 0/1***

2. Suppose you have coins of denominations 1,3 and 4. You use a greedy algorithm, in which you choose the largest denomination coin which is not greater than the remaining sum. For which of the following sums, will the algorithm doesn't produce an optimal answer?

Answer

10

vtu29414

Status : Correct ***Marks : 1/1***

3. Which data structure is most commonly used to efficiently implement a greedy algorithm that repeatedly selects the minimum (or maximum) element at each step?

Answer

Heap

Status : Correct ***Marks : 1/1***

4. What is a greedy algorithm?

Answer

An algorithm that makes the best choice at each step, without considering the overall impact

Status : Correct ***Marks : 1/1***

5. Which of the following problems is typically solved using a greedy algorithm?

Answer

Coin change problem.

Status : Wrong ***Marks : 0/1***

6. What is the primary advantage of greedy algorithms over other approaches for certain problems?

Answer

Greedy algorithms are easy to implement

Status : Correct ***Marks : 1/1***

7. Which of the following is a disadvantage of greedy algorithm?

Answer

It may not always find the globally optimal solution

Status : Correct ***Marks : 1/1***

8. Which of the following approaches should be used to find the solution to the activity selection problem?

Answer

Greedy approach

Status : Correct ***Marks : 1/1***

9. Suppose two activities A and B, have start and finish times as SA, FA and SB, FB respectively. Both activities are said to be compatible, under which of the following conditions?

Answer

SA >= FB or SB >= FA

Status : Correct ***Marks : 1/1***

10. In the activity selection problem, if two activities have the same finish time, which one is selected?

Answer

The one with the earlier start time

Status : Correct ***Marks : 1/1***

11. Consider the following number of activities with their start and finish time given below. In which sequence will the activity be selected in order to maximize the number of activities, without any conflicts?

Answer

A1, A2, A5, A6

Status : Wrong ***Marks : 0/1***

12. Which of the following is a characteristic of the greedy algorithm?

Answer

It makes the best choice at each step by considering the global problem state.

Status : Wrong ***Marks : 0/1***

13. What is the primary reason why the greedy algorithm does not work for some optimization problems?

Answer

It does not reconsider previous decisions and makes choices based only on the current state.

Status : Correct ***Marks : 1/1***

14. In the greedy approach using a stack, what condition do we check before popping from the stack

Answer

If the top is greater than current digit and k > 0

Status : Correct ***Marks : 1/1***

15. Why is sorting not used in this greedy solution?

Answer

We need to maintain digit order

Status : Wrong ***Marks : 0/1***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 11_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Given a set of coin denominations and a target value V, determine the minimum number of coins required to make the value V.

If it's not possible to make the value with the given coins, return -1.

Input Format

The first line contains an integer n, representing the number of coin denominations.

The second line contains n space-separated integers, representing the coin denominations.

The third line contains an integer amount, representing the target value V.

Output Format

vtu29414

The output prints the minimum number of coins required to make the target value.

Print -1 if the value cannot be formed using the given denominations.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 5
11
Output: 3

Answer

```
// You are using GCC
#include <stdio.h>

#define INF 1000000

int min(int a, int b) {
    return (a < b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int coins[15];

    for (int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }

    int amount;
    scanf("%d", &amount);

    int dp[105];

    for (int i = 0; i <= amount; i++)
```

```
        dp[i] = INF;

    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i) {
                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
            }
        }
    }

    if (dp[amount] == INF)
        printf("-1");
    else
        printf("%d", dp[amount]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement:

Rajesh runs a delivery company and needs to deliver n packages to different cities. Each package i has:

A weight wt[i]A profit val[i] if delivered on timeA delivery deadline deadline[i] (in hours from now)Rajesh has a truck with maximum weight capacity W and can only deliver one package at a time. He can choose the order of deliveries, but if a package is delivered after its deadline, he earns zero profit for it.

Task: Determine the maximum total profit Rajesh can earn by selecting which packages to deliver and in what order, without exceeding the truck's capacity at any point.

Input Format

First line: integer n — number of packages

Second line: n integers — values of each package (profit)

Third line: n integers — weights of each package

Fourth line: n integers — deadlines of each package

Fifth line: integer W — truck capacity

Output Format

The output prints the single integer maximum total profit.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
100 200 150 120
10 20 15 15
5 3 4 2
30

Output: 470

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

typedef struct {
    int val;
    int wt;
    int dl;
} Item;

// comparator for sorting by deadline
int cmp(const void *a, const void *b) {
    Item *x = (Item *)a;
```

```
    Item *y = (Item *)b;
    return x->dl - y->dl;
}

// check if subset is feasible
int isValid(Item arr[], int n, int mask, int W) {
    Item temp[25];
    int k = 0, totalW = 0;

    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) {
            temp[k++] = arr[i];
            totalW += arr[i].wt;
        }
    }

    if (totalW > W) return 0;

    qsort(temp, k, sizeof(Item), cmp);

    int time = 0;
    for (int i = 0; i < k; i++) {
        time++;
        if (time > temp[i].dl)
            return 0;
    }

    return 1;
}

int main() {
    int n;
    scanf("%d", &n);

    Item arr[25];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i].val);

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i].wt);
```

```
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i].dl);

    int W;
    scanf("%d", &W);

    int maxProfit = 0;

    int total = (1 << n);

    for (int mask = 0; mask < total; mask++) {
        int profit = 0;

        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                profit += arr[i].val;
            }
        }

        if (isValid(arr, n, mask, W)) {
            maxProfit = max(maxProfit, profit);
        }
    }

    printf("%d", maxProfit);

    return 0;
}
```

Status : Wrong **Marks : 0/10**

3. Problem Statement

You are given n items, each with a certain weight and value. Your task is to select items to put in a knapsack of capacity W so that the total value is maximized. You cannot break an item; you must either take the whole item or leave it.

Formally, given two integer arrays, val[0..n-1] and wt[0..n-1], representing the values and weights of n items respectively, and an integer W

representing the knapsack capacity, determine the maximum value subset of items such that the total weight does not exceed W.

Solve this problem using dynamic programming.

Input Format

The first line contains an integer n, the number of items.

The second line contains n integers, the values of each item separated by spaces.

The third line contains n integers, the weights of each item separated by spaces.

The fourth line contains an integer W, the maximum capacity of the knapsack.

Output Format

Print a single integer representing the maximum value that can be carried in the knapsack.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2
50 60
5 10
10
Output: 60

Answer

```
#include <stdio.h>

int max(int a, int b) {
   return (a > b) ? a : b;
}

int main() {
   int n;
   scanf("%d", &n);
```

```
    int val[105], wt[105];

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    int W;
    scanf("%d", &W);

    int dp[105][1005] = {0};

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {

            if (wt[i - 1] <= w) {
                dp[i][w] = max(dp[i - 1][w],
                               val[i - 1] + dp[i - 1][w - wt[i - 1]]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    printf("%d", dp[n][W]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Imagine you are a tourist in Moscow and have booked a bus tour to see some amazing attractions just outside of town. The bus first drives around town for a while (a long while, since Moscow is a big city) picking up people at their respective hotels. It then proceeds to the amazing attraction, and after a few hours go back into the city, again driving to each

hotel, this time to drop people off.

For some reason, whenever you do this, your hotel is always the first to be visited for pickup and the last to be visited for drop-off, meaning that you have to suffer through two not-so-amazing sightseeing tours of all the local hotels.

This is clearly not what you want to do (unless for some reason you are really into hotels), so let's fix it. We will develop some software to enable the sightseeing company to route its bus tours more fairly—though it may sometimes mean a longer total distance for everyone, fair is fair, right?

For this problem, there is a starting location (the sightseeing company headquarters), h hotels that need to be visited for pickups and dropoffs, and a destination location (the amazing attraction). We need to find a route that goes from the headquarters, through all the hotels, to the attraction, then back through all the hotels again (possibly in a different order), and finally back to the headquarters.

To guarantee that none of the tourists (and, in particular, you) is forced to suffer through two full tours of the hotels, we require that every hotel that is visited among the first $h/2$ hotels on the way to the attraction is also visited among the first $h/2$ hotels on the way back.

Subject to these restrictions, we would like to make the complete bus tour as short as possible. Note that these restrictions may force the bus to drive past a hotel without stopping there (this is not considered visiting) and then visit it later, as illustrated in the sample input.

Input Format

The first line of each test case consists of two integers n and m, where n is the number of locations (hotels, headquarters, attraction) and m is the number of pairs of locations between which the bus can travel.

The n different locations are numbered from 0 to n−1, where 0 is the headquarters, 1 through n−2 are the hotels, and n−1 is the attraction. Assume that there is at most one direct connection between any pair of locations and it is possible to travel from any location to any other location (but not necessarily directly).

Following the first line are m lines, each containing three integers u, v, and t, indicating that the bus can go directly between locations u and v in t seconds (in either direction).

Output Format

For each test case, print the case number and the minimum total travel time of the complete tour.

The output should follow this format: "Case X: Y"

where:

X is the test case number starting from 1.

Y is the shortest possible total time (in seconds) for the bus tour.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 4
0 1 10
1 2 20
2 3 30
3 4 40
4 6
0 1 1
0 2 1
0 3 1
1 2 1
1 3 1
2 3 1

Output: Case 1: 300
Case 2: 6

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
```

```
#define INF 1000000000

int min(int a, int b) {
    return (a < b) ? a : b;
}

int dist[25][25];
int dp[1 << 20][20];

int main() {
    int n, m;
    int tc = 1;

    while (scanf("%d %d", &n, &m) != EOF) {

        // initialize distances
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = (i == j) ? 0 : INF;
            }
        }

        for (int i = 0; i < m; i++) {
            int u, v, t;
            scanf("%d %d %d", &u, &v, &t);
            dist[u][v] = min(dist[u][v], t);
            dist[v][u] = min(dist[v][u], t);
        }

        // Floyd-Warshall
        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }

        int FULL = (1 << n);
```

```
    for (int i = 0; i < FULL; i++)
        for (int j = 0; j < n; j++)
            dp[i][j] = INF;

    dp[1][0] = 0; // start at headquarters (0)

    for (int mask = 0; mask < FULL; mask++) {
        for (int u = 0; u < n; u++) {
            if (dp[mask][u] == INF) continue;

            for (int v = 0; v < n; v++) {
                if (!(mask & (1 << v))) {
                    int newMask = mask | (1 << v);
                    if (dist[u][v] != INF) {
                        dp[newMask][v] = min(dp[newMask][v],
                                             dp[mask][u] + dist[u][v]);
                    }
                }
            }
        }
    }

    int ans = INF;

    for (int i = 0; i < n; i++) {
        if (dist[i][n - 1] != INF) {
            ans = min(ans, dp[FULL - 1][i] + dist[i][n - 1] + dist[n - 1][0]);
        }
    }

    printf("Case %d: %d ", tc++, ans);
  }

  return 0;
}
```

Status : Wrong ***Marks : 0/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 11_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an integer n, representing the total number of stairs, the task is to determine the number of distinct ways to reach the top when at each move you can climb 1, 2, or 3 steps.

Write a program using the Dynamic Programming (Bottom-Up Approach) algorithm to ensure that the total number of possible ways to climb the stairs is correctly calculated and displayed.

Examples:

Input: 4

Output: 7

Explanation: There are seven ways: {1, 1, 1, 1}, {1, 2, 1}, {2, 1, 1}, {1, 1, 2}, {2,

2}, {3, 1}, {1, 3}.

Input: 3

Output: 4

Explanation: There are four ways: {1, 1, 1}, {1, 2}, {2, 1}, {3}.

Input Format

The input contains a single integer n, representing the total number of stairs.

Output Format

The output prints a single integer representing the total number of distinct ways to reach the top.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
Output: 7

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int dp[25];

    dp[0] = 1;

    if (n >= 1) dp[1] = 1;
    if (n >= 2) dp[2] = 2;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3];
    }
```

```
    printf("%d", dp[n]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Given a non-negative integer n, the task is to compute the n-th Lucas number using the Dynamic Programming (Bottom-Up Approach) algorithm. The Lucas sequence follows the recurrence relation:

L(0) = 2, L(1) = 1, and L(n) = L(n−1) + L(n−2) for n ≥ 2.

Write a program to ensure that the n-th Lucas number is correctly calculated and displayed.

Input Format

The input contains a single integer n, representing the position in the Lucas sequence.

Output Format

The output prints a single integer representing the n-th Lucas number.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
Output: 4

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int n;
    scanf("%d", &n);

    int dp[25];

    dp[0] = 2;
    if (n >= 1)
        dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    printf("%d", dp[n]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement:

Dr. Ananya is a computational biologist working on analyzing DNA sequences. She needs to identify the longest palindromic subsequence within a DNA string, which may represent certain symmetric patterns important for genetic research.

Given a string S representing a DNA sequence (composed of letters A, C, G, T), find the length of the longest subsequence of S that reads the same forwards and backwards.

Note:

A subsequence is formed by deleting zero or more characters without changing the order of the remaining characters.The DNA sequences can be long, and an efficient solution using dynamic programming is required to handle them within computational limits.

Input Format

The input displays a single line containing the DNA string S.

Output Format

The output displays a single integer representing the length of the longest palindromic subsequence in S.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: madam

Output: 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char s[105];
    scanf("%s", s);

    int n = strlen(s);
    int dp[105][105];

    // Every single character is a palindrome of length 1
    for (int i = 0; i < n; i++) {
        dp[i][i] = 1;
    }

    // Build table for substrings of increasing length
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;

            if (s[i] == s[j]) {
```

```
            if (len == 2)
                dp[i][j] = 2;
            else
                dp[i][j] = dp[i + 1][j - 1] + 2;
        } else {
            dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
        }
    }
  }

  printf("%d", dp[0][n - 1]);

  return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Arjun is tracking the stock prices for the last N days. He wants to invest in a way that he buys and sells stocks alternately to maximize his profit. He can either buy or skip on a day, but after buying he must sell before buying again. Find the maximum profit he can achieve from the stock prices. Help him to implement the task.

Formula

Let dp[i][0] = maximum profit on day i if not holding a stock

Let dp[i][1] = maximum profit on day i if holding a stock

Transition:

dp[i][0] = max(dp[i-1][0], dp[i-1][1] + prices[i]) either skip or sell today

dp[i][1] = max(dp[i-1][1], dp[i-1][0] - prices[i]) either keep holding or buy today

Base case:

dp[0][0] = 0

dp[0][1] = -prices[0]

Result:

max_profit = dp[N-1][0]

Input Format

The first line of input consists of Integer N representing the number of days.

The second line of input consists of N Integers prices[i] representing the stock price on day i.

Output Format

The output prints: "Maximum profit:" max_profit

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: Maximum profit: 2

Answer

```
// You are using Java
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int prices[1005];

    for (int i = 0; i < n; i++) {
        scanf("%d", &prices[i]);
    }
```

```
    int dp0 = 0;            // dp[i][0]
    int dp1 = -prices[0];     // dp[i][1]

    for (int i = 1; i < n; i++) {
        int new_dp0 = max(dp0, dp1 + prices[i]);  // sell or skip
        int new_dp1 = max(dp1, dp0 - prices[i]);  // buy or hold

        dp0 = new_dp0;
        dp1 = new_dp1;
    }

    printf("Maximum profit: %d", dp0);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

5. Problem Statment

To celebrate Diwali, Dhoni wants to buy a variety of crackers. He noticed four different types of crackers on the menu list, each with a price. He's now trying to figure out how many different ways he can get the crackers for Rs.N.

Rockets - 1 RsSparklers - 2 RsChakkars - 3 RsFlower pots - 5 Rs

Example

Input

4

Output

The total number of ways to get crackers is 4

Explanation

1st way -> 4 rockets

2nd way -> 2 Sparklers

3rd way -> 1 Sparkler, 2 rockets

4th way -> 1 Chakkar, 1 rocket

Input Format

The input consists of a single integer N, representing the total amount (in rupees) Dhoni wants to spend on crackers.

Output Format

The output prints "The total number of ways to get crackers is X" Where X representing the total number of different ways Dhoni can buy crackers such that the total cost equals N.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

Output: The total number of ways to get crackers is 6

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    long long dp[n + 1];

    for (int i = 0; i <= n; i++)
        dp[i] = 0;

    dp[0] = 1;

    int coins[] = {1, 2, 3, 5};

    for (int c = 0; c < 4; c++) {
        int coin = coins[c];
        for (int i = coin; i <= n; i++) {
            dp[i] += dp[i - coin];
```

```
    }
    }

    printf("The total number of ways to get crackers is %lld", dp[n]);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 11_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 12

Section 1 : MCQ

1. Consider the following two sequences:

X = < B, C, D, C, A, B, C >Y = < C, A, D, B, C, B >

The length of longest common subsequence of X and Y is:

Answer

4

Status : Correct ***Marks : 1/1***

2. Which of the following problems should be solved using dynamic programming?

Answer

Longest common subsequence

vtu29414

Status : Correct ***Marks : 1/1***

3. Which of the following problems is closely related to the Subsets Sum problem and can also be solved using dynamic programming?

Answer

Coin Change Problem

Status : Correct ***Marks : 1/1***

4. Consider two strings A = "qpqrr" and B = "pqprqrp". Let x be the length of the longest common subsequence (not necessarily contiguous) between A and B and let y be the number of such longest common subsequences between A and B.

Then x + 10y = _____.

Answer

34

Status : Correct ***Marks : 1/1***

5. Consider the strings "PQRSTPQRS" and "PRATPBRQRPS". What is the length of the longest common subsequence?

Answer

6

Status : Wrong ***Marks : 0/1***

6. Which of the following is/are property/properties of a dynamic programming problem?

Answer

Both optimal substructure and overlapping subproblems

Status : Correct ***Marks : 1/1***

7. If an optimal solution can be created for a problem by constructing optimal solutions for its subproblems, the problem possesses _____ property.

Answer

Optimal substructure

Status : Correct ***Marks : 1/1***

8. In dynamic programming, the technique of storing the previously calculated values is called _____.

Answer

Memoization

Status : Correct ***Marks : 1/1***

9. Which of the following is an example of a problem that can be solved using memoization in dynamic programming?

Answer

Computing the edit distance between two strings

Status : Correct ***Marks : 1/1***

10. When a top-down approach to dynamic programming is applied to a problem, it usually _____.

Answer

decreases the time complexity and increases the space complexity

Status : Correct ***Marks : 1/1***

11. Which of the following problems is NOT solved using dynamic programming?

Answer

vtu29414

Fractional knapsack problem

Status : Correct ***Marks : 1/1***

12. Which of the following is NOT a characteristic of dynamic programming?

Answer

Solving problems in a sequential manner.

Status : Correct ***Marks : 1/1***

13. What happens when a top-down approach of dynamic programming is applied to any problem?

Answer

It increases the space complexity and decreases the time complexity.

Status : Correct ***Marks : 1/1***

14. The Fibonacci sequence is often used to illustrate dynamic programming concepts. What is the time complexity of a naive recursive implementation of Fibonacci numbers?

Answer

O(log n)

Status : Wrong ***Marks : 0/1***

15. What are the different techniques to solve dynamic programming problems:

Answer

Memoization

Status : Wrong ***Marks : 0/1***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 10_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Sai has been studying various algorithms related to heaps. Recently, he came across a problem where he needed to filter elements from a given array based on certain criteria related to heap properties.

He wants to build a max heap from a given array of integers, delete elements that fall outside the specified range, and then rearrange the elements in the form of a Max Heap.

Write a program to help Sai in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

vtu29414

The second line consists of N space-separated integers, representing the elements of the array.

The third line consists of two integers, a and b, separated by a space, representing the range to filter the elements.

Output Format

The output prints the space-separated integers, representing the elements that remain in the max heap after removing elements outside the given range.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 7 8 3 12
1 6

Output: 5 3

Answer

```
# You are using Python
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)


def build_max_heap(arr):
    n = len(arr)
```

```
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)


# Input
data = list(map(int, input().split()))
N = data[0]
arr = data[1:N+1]
a, b = data[N+1], data[N+2]

# Step 1: Build max heap
build_max_heap(arr)

# Step 2: Filter elements within range [a, b]
filtered = [x for x in arr if a <= x <= b]

# Step 3: Rebuild max heap from filtered elements
build_max_heap(filtered)

# Output
print(*filtered)
```

Status : Wrong ***Marks : 0/10***

2. Problem Statement

Maxwell, an enthusiastic computer science student, is exploring the fascinating world of data structures. His task is to create a program that constructs a min heap from a list of integers and identifies negative numbers within it.

Help Maxwell in completing the program.

Input Format

The first line of input consists of an integer N, representing the number of integers in the array.

The second line consists of N space-separated integers, representing the elements of the list.

Output Format

The first line of output prints the min-heap as a space-separated list of integers.

The second line prints the negative numbers, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
15 -5 20 -19 23 42 29
Output: -19 -5 20 15 23 42 29
-19 -5

Answer

```
// You are using GCC
#include <stdio.h>

// Function to heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        int temp = arr[i];
        arr[i] = arr[smallest];
        arr[smallest] = temp;

        heapify(arr, n, smallest);
    }
}
```

```
// Function to build min heap
void buildMinHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[10];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Build Min Heap
    buildMinHeap(arr, n);

    // Print Min Heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    // Print negative numbers
    for (int i = 0; i < n; i++) {
        if (arr[i] < 0) {
            printf("%d ", arr[i]);
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

You are given an array of n integers. You need to perform q queries of two types:

Q l r — Find the minimum value in the subarray from index l to r (inclusive).U idx val — Update the element at index idx to val.

Implement an efficient solution using a Segment Tree to handle both query and update operations.

Input Format

The first line contains two space-separated integers n and q, representing the size of the array and the number of queries

The second line contains n space-separated integers representing the elements of the array.

Next q lines: Each line contains a query in one of the following formats:

- Q l r
- U idx val

Output Format

For each query of type Q, print the minimum value in the specified range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3 2
4 2 5
Q 0 2
Q 1 1

Output: 2
2

Answer

```
// You are using GCC
#include <stdio.h>

// Function to heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
```

```
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        int temp = arr[i];
        arr[i] = arr[smallest];
        arr[smallest] = temp;

        heapify(arr, n, smallest);
    }
}

// Function to build min heap
void buildMinHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[10];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Build Min Heap
    buildMinHeap(arr, n);

    // Print Min Heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
```

```
    }

    // Print negative numbers
    for (int i = 0; i < n; i++) {
        if (arr[i] < 0) {
            printf("%d ", arr[i]);
        }
    }

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

4. Problem Statement

Meera is working on a text processing task where she wants to identify the longest common prefix among a given set of words. She decides to use a Trie structure to efficiently compute this prefix.

Write a program to help Meera insert the given words into a Trie and then determine the longest common prefix among them.

Input Format

The first line contains an integer n, representing the number of words.

The next n lines each contain a single word.

Output Format

The output prints "The longest common prefix is: <prefix>" if a common prefix exists.

Otherwise, print "There is no common prefix"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
aeroplane
aerospace
aerial
aesthetic
Output: The longest common prefix is: ae

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ALPHABET_SIZE 26

// Trie node structure
struct TrieNode {
    struct TrieNode* children[ALPHABET_SIZE];
    int isEnd;
};

// Create a new Trie node
struct TrieNode* createNode() {
    struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
    node->isEnd = 0;
    for (int i = 0; i < ALPHABET_SIZE; i++)
        node->children[i] = NULL;
    return node;
}

// Insert word into Trie
void insert(struct TrieNode* root, char* word) {
    struct TrieNode* curr = root;
    for (int i = 0; word[i]; i++) {
        int index = word[i] - 'a';
        if (!curr->children[index])
            curr->children[index] = createNode();
        curr = curr->children[index];
    }
    curr->isEnd = 1;
}
```

```
// Count children of a node
int countChildren(struct TrieNode* node, int* index) {
    int count = 0;
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        if (node->children[i]) {
            count++;
            *index = i;
        }
    }
    return count;
}

// Find longest common prefix
void longestCommonPrefix(struct TrieNode* root, char* prefix) {
    struct TrieNode* curr = root;
    int index;
    while (countChildren(curr, &index) == 1 && curr->isEnd == 0) {
        prefix[strlen(prefix)] = 'a' + index;
        prefix[strlen(prefix) + 1] = '\0';
        curr = curr->children[index];
    }
}

int main() {
    int n;
    scanf("%d", &n);

    struct TrieNode* root = createNode();
    char words[10][101];

    for (int i = 0; i < n; i++) {
        scanf("%s", words[i]);
        insert(root, words[i]);
    }

    char prefix[101] = "";
    longestCommonPrefix(root, prefix);

    if (strlen(prefix) > 0)
        printf("The longest common prefix is: %s", prefix);
    else
        printf("There is no common prefix");
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 10_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Jovika is learning about heaps and wants to implement a program that constructs a max heap and inserts elements into it.

She wants to write a program that implements max heap insertion. The program defines functions to swap elements and perform maximum heapification. She wants to extend this program to take a list of integers, build a max heap, and insert a new element into the heap.

Help her in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements to be inserted into the max heap.

vtu29414

The second line consists of N space-separated integers, representing the elements to be inserted into the max heap.

The third line consists of an integer representing the new element to be inserted into the max heap.

Output Format

The output prints the space-separated integers, representing the elements in the max heap after inserting the new element in the order of insertion.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
3 1 4 2 5
6
Output: 6 3 5 2 1 4

Answer

```
// You are using
#include <stdio.h>

#define MAX 20

// Swap function
void swap(int *a, int *b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}

// Heapify (top-down)
void heapify(int arr[], int n, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   if (left < n && arr[left] > arr[largest])
```

```
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// Build max heap
void buildMaxHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

// Insert element into heap
void insert(int arr[], int *n, int value) {
    (*n)++;
    arr[*n - 1] = value;

    int i = *n - 1;

    // Heapify up
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (arr[parent] < arr[i]) {
            swap(&arr[parent], &arr[i]);
            i = parent;
        } else {
            break;
        }
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[MAX];
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int newElement;
    scanf("%d", &newElement);

    // Step 1: Build max heap
    buildMaxHeap(arr, n);

    // Step 2: Insert new element
    insert(arr, &n, newElement);

    // Output heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Bob is fascinated by numbers and loves arranging them in various ways. His recent exploration led him to Max Heaps.

Bob likes odd numbers more than even numbers. After inserting elements into a Max Heap, he wants to remove all even numbers from it while keeping the Max Heap property intact.

Your task is to help Bob build a Max Heap from the given elements, display it, delete all the even numbers from the resultant array and then rearrange the elements in the form of Max Heap.

Input Format

The first line of input consists of an integer N, representing the number of elements initially to be inserted into the Max Heap.

The second line consists of N space-separated integers, representing the elements to be inserted into the heap.

Output Format

The first line of output prints the space-separated elements of the Max Heap after building it from the given N elements.

The second line prints the space-separated elements of the Max Heap after removing even elements.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
3 2 1

Output: 3 2 1
3 1

Answer

```
// You are using GCC
#include <stdio.h>

// Function to heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i;      // Initialize largest as root
    int left = 2 * i + 1;  // left child
    int right = 2 * i + 2; // right child

    // If left child is larger
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // If right child is larger
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is not root
    if (largest != i) {
```

```
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        // Recursively heapify the affected subtree
        heapify(arr, n, largest);
    }
}

// Function to build Max Heap
void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

// Function to print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Build Max Heap
    buildHeap(arr, n);

    // Step 2: Print Max Heap
    printArray(arr, n);
    printf(" ");

    // Step 3: Remove even numbers
    int oddArr[20];
```

```
    int newSize = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] % 2 != 0) {
            oddArr[newSize++] = arr[i];
        }
    }

    // Step 4: Rebuild heap with odd numbers
    buildHeap(oddArr, newSize);

    // Step 5: Print new heap
    printArray(oddArr, newSize);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Iniya is studying data structures and algorithms, and she is currently learning about heaps.

She wants to practice implementing Min-Heap and perform a deletion operation on it. A min-heap is a binary tree in which the value of each node is less than or equal to the values of its children.

Write a program to assist Iniya in building a min-heap and delete the root element.

Input Format

The first line of input consists of an integer N, representing the number of elements in the initial Min-Heap.

The second line consists of N space-separated integers, where each integer represents an element in the Min-Heap.

Output Format

The output prints the space-separated elements of the Min-Heap after

performing the deletion operation of the root.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
80 40 30 10 50 33 66

Output: 30 40 33 80 50 66

Answer

```
// You are using GCC
#include <stdio.h>

// Heapify function for Min Heap
void heapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    // Check left child
    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    // Check right child
    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    // Swap and continue heapifying if needed
    if (smallest != i) {
        int temp = arr[i];
        arr[i] = arr[smallest];
        arr[smallest] = temp;

        heapify(arr, n, smallest);
    }
}

// Build Min Heap
void buildMinHeap(int arr[], int n) {
```

```
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Build Min Heap
    buildMinHeap(arr, n);

    // Step 2: Delete root
    arr[0] = arr[n - 1];
    n--;

    // Step 3: Heapify root
    heapify(arr, n, 0);

    // Step 4: Print result
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

A trie (pronounced as "try") or prefix tree is a tree data structure used to store and retrieve keys in a dataset of strings efficiently. This data structure has various applications, such as autocomplete and spellchecker.

Implement the Trie class:

Trie() Initializes the trie object.void insert(String word): Inserts the string word into the trie.boolean search(String word) Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.boolean startsWith(String prefix) Returns true if a previously inserted string word has the prefix, and false otherwise.

Example

Input:

4

apple

cat

orange

mat

cat

ora

Output:

The word is present in the trie.

There is a word with the given prefix in the trie.

Input Format

The first line of the input consists of the number of words to insert, N.

The next N lines of input consist of the words in each line.

The next line of input consists of the word to search.

The last line of the input consists of the prefix to search.

Output Format

When searching for a word using the search method, the program will output one of the following messages:

- If the word is present in the trie, the output prints "The word is present in the trie."
- If the word is not present in the trie, the output prints "The word is not present in the trie."

When checking for a prefix using the startsWith method, the program will output one of the following messages:

- If there is a word in the trie with the given prefix, the output prints "There is a word with the given prefix in the trie."
- If there is no word in the trie with the given prefix, the output prints "There is no word with the given prefix in the trie."

Sample Test Case

Input: 4
apple
cat
orange
mat
catlyn
ora

Output: The word is not present in the trie.
There is a word with the given prefix in the trie.

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define ALPHABET_SIZE 26

// Trie Node
struct TrieNode {
    struct TrieNode* children[ALPHABET_SIZE];
    bool isEndOfWord;
};
```

```
// Create a new Trie node
struct TrieNode* createNode() {
    struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
    node->isEndOfWord = false;
    for (int i = 0; i < ALPHABET_SIZE; i++)
        node->children[i] = NULL;
    return node;
}

// Insert a word into Trie
void insert(struct TrieNode* root, char* word) {
    struct TrieNode* curr = root;
    for (int i = 0; word[i]; i++) {
        int index = word[i] - 'a';
        if (!curr->children[index])
            curr->children[index] = createNode();
        curr = curr->children[index];
    }
    curr->isEndOfWord = true;
}

// Search full word
bool search(struct TrieNode* root, char* word) {
    struct TrieNode* curr = root;
    for (int i = 0; word[i]; i++) {
        int index = word[i] - 'a';
        if (!curr->children[index])
            return false;
        curr = curr->children[index];
    }
    return curr->isEndOfWord;
}

// Check prefix
bool startsWith(struct TrieNode* root, char* prefix) {
    struct TrieNode* curr = root;
    for (int i = 0; prefix[i]; i++) {
        int index = prefix[i] - 'a';
        if (!curr->children[index])
            return false;
        curr = curr->children[index];
    }
```

```
    return true;
}

int main() {
    int n;
    scanf("%d", &n);

    struct TrieNode* root = createNode();

    char word[100];

    // Insert words
    for (int i = 0; i < n; i++) {
        scanf("%s", word);
        insert(root, word);
    }

    char searchWord[100], prefix[100];

    scanf("%s", searchWord);
    scanf("%s", prefix);

    // Search word
    if (search(root, searchWord))
        printf("The word is present in the trie.\n");
    else
        printf("The word is not present in the trie.\n");

    // Check prefix
    if (startsWith(root, prefix))
        printf("There is a word with the given prefix in the trie.");
    else
        printf("There is no word with the given prefix in the trie.");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Given an array of integers representing the combat power of N players.

You need to efficiently perform the following operations:

Query (Q L R): Find the maximum combat power between ranks L and R (inclusive).Update (U i val): Update the power of the player at index i to a new value val.

To ensure efficiency, implement these operations using a Segment Tree.

Input Format

The first line contains two space-separated integers, N and Q, representing the number of players and the number of operations.

The second line contains N space-separated integers representing the initial combat power scores of all players, where index 0 is the top-ranked player.

The next Q lines describe the operations:

- "Q L R" Query represents the maximum combat power from ranks L to R (inclusive).
- "U i val" Updates the power score of the player at rank i to val.

All indices are 0-based.

Output Format

For each query Q L R, print the maximum combat power in that range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 5
4 7 2 9 5 1
Q 1 4
U 3 6
Q 1 4
U 0 10

vtu29414

Q 0 2
Output: 9
7
10

Answer

```
// You are using GCC
#include <stdio.h>

#define MAX 100

int arr[MAX];
int seg[4 * MAX];

// Build Segment Tree
void build(int index, int low, int high) {
    if (low == high) {
        seg[index] = arr[low];
        return;
    }
    int mid = (low + high) / 2;
    build(2 * index + 1, low, mid);
    build(2 * index + 2, mid + 1, high);

    // Store maximum
    seg[index] = seg[2 * index + 1] > seg[2 * index + 2]
                ? seg[2 * index + 1]
                : seg[2 * index + 2];
}

// Range Maximum Query
int query(int index, int low, int high, int l, int r) {
    // No overlap
    if (r < low || high < l)
        return -1;

    // Complete overlap
    if (l <= low && high <= r)
        return seg[index];

    // Partial overlap
    int mid = (low + high) / 2;
```

```
    int left = query(2 * index + 1, low, mid, l, r);
    int right = query(2 * index + 2, mid + 1, high, l, r);

    return (left > right) ? left : right;
}

// Update value
void update(int index, int low, int high, int i, int val) {
    if (low == high) {
        arr[i] = val;
        seg[index] = val;
        return;
    }

    int mid = (low + high) / 2;
    if (i <= mid)
        update(2 * index + 1, low, mid, i, val);
    else
        update(2 * index + 2, mid + 1, high, i, val);

    seg[index] = seg[2 * index + 1] > seg[2 * index + 2]
                ? seg[2 * index + 1]
                : seg[2 * index + 2];
}

int main() {
    int n, q;
    scanf("%d %d", &n, &q);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Build tree
    build(0, 0, n - 1);

    while (q--) {
        char type;
        scanf(" %c", &type);

        if (type == 'Q') {
            int l, r;
```

```
        scanf("%d %d", &l, &r);
        printf("%d\n", query(0, 0, n - 1, l, r));
    }
    else if (type == 'U') {
        int i, val;
        scanf("%d %d", &i, &val);
        update(0, 0, n - 1, i, val);
    }
  }

  return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 10_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 10

Section 1 : MCQ

1. Which of the following sequences of array elements forms a heap?

Answer

{23, 17, 14, 6, 13, 10, 1, 12, 7, 5}

Status : Wrong ***Marks : 0/1***

2. Consider a max heap, represented by the array: 23, 17, 14, 6, 13, 10, 1, 5. If a value 20 is inserted into this heap, then what will be the new max heap?

Answer

23, 20, 14, 17, 13, 10, 1, 5, 6

Status : Correct ***Marks : 1/1***

vtu29414

3. Given a binary max heap. The elements are sorted in an array as 25, 14, 16, 13, 10, 8, 12. What is the content of the array after two delete operations?

Answer

14, 13, 12, 8, 10

Status : Correct ***Marks : 1/1***

4. In a Min Heap, what is the condition that must be satisfied between a parent and its children?

Answer

The parent must have a lower value than its children

Status : Correct ***Marks : 1/1***

5. When inserting an element with a value of 8 into a Min Heap, where would it be placed if the current heap contains the elements [10, 12, 15, 20, 17]?

Answer

As the left child of the root

Status : Wrong ***Marks : 0/1***

6. If you're inserting an element with a value of 5 into a Min Heap, and the current heap contains the elements [11, 16, 10, 13, 18], what will the Min Heap look like after the insertion?

Answer

5, 10, 11, 13, 16, 18

Status : Wrong ***Marks : 0/1***

7. When deleting the root element from a Min Heap, which of the following replaces it as the new root?

Answer

the last element from the tree

Status : Correct ***Marks : 1/1***

8. Tries are mainly used for:

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

9. In a trie, how is the end of a valid word usually represented?

Answer

By using an end-of-word (terminal) marker at a node

Status : Correct ***Marks : 1/1***

10. Which of the following is true about tries?

Answer

All of the mentioned options

Status : Correct ***Marks : 1/1***

11. What is stored in a segment tree node for a Range Minimum Query (RMQ)?

Answer

Minimum element in range

Status : Correct ***Marks : 1/1***

12. In a Range Max Query tree, what is the internal node value?

Answer

Minimum of child values

Status : Wrong ***Marks : 0/1***

13. Which segment tree operation is used for range max queries?

Answer

max() merge

Status : Correct ***Marks : 1/1***

14. What data structure is often used in segment tree nodes to count distinct elements?

Answer

Queue

Status : Wrong ***Marks : 0/1***

15. In range distinct count queries, what does each segment tree node contain?

Answer

Set of elements

Status : Correct ***Marks : 1/1***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 9_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be moved.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j, representing the column index of the fixed queen.

Output Format

If solutions exist: Print each valid board configuration, where "Q" represents a queen and "." represents an empty cell.

Each row of the board should be space-separated.

Each configuration should be followed by a blank line.

If no configuration is possible, print "No solution".

Refer to the sample output for the formatting specification.

Sample Test Case

Input: 4
0
1
Output: . Q . .
. . . Q
Q . . .
. . Q .

Answer

// You are using GCC#include <stdio.h>

int n;

```
int board[20][20];
int col[20], diag1[40], diag2[40];
int fx, fy;
int found = 0;

void printBoard() {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (board[i][j])
                printf("Q ");
            else
                printf(". ");
        }
        printf(" ");
    }
    printf(" ");
}

void solve(int r) {
    if (r == n) {
        printBoard();
        found = 1;
        return;
    }

    // skip fixed row
    if (r == fx) {
        if (!col[fy] && !diag1[r + fy] && !diag2[r - fy + n]) {
            col[fy] = diag1[r + fy] = diag2[r - fy + n] = 1;
            board[r][fy] = 1;

            solve(r + 1);

            board[r][fy] = 0;
            col[fy] = diag1[r + fy] = diag2[r - fy + n] = 0;
        }
        return;
    }

    for (int c = 0; c < n; c++) {
        if (!col[c] && !diag1[r + c] && !diag2[r - c + n]) {
            board[r][c] = 1;
```

```
            col[c] = diag1[r + c] = diag2[r - c + n] = 1;

            solve(r + 1);

            board[r][c] = 0;
            col[c] = diag1[r + c] = diag2[r - c + n] = 0;
        }
    }
}

int main() {
    scanf("%d", &n);
    scanf("%d %d", &fx, &fy);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            board[i][j] = 0;

    solve(0);

    if (!found)
        printf("No solution");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Consider a rat placed at (0, 0) in a square matrix of order N*N. It has to reach the destination at (N – 1, N – 1). Find all possible paths that the rat can take to reach from the source to the destination. The directions in which the rat can move are 'U'(up), 'D'(down), 'L' (left), and 'R' (right). Value 0 at a cell in the matrix represents that it is blocked and the rat cannot move to it while value 1 at a cell in the matrix represents that the rat can travel through it. Return the list of paths in lexicographically increasing order.

Note: In a path, no cell can be visited more than once. If the source cell is

0, the rat cannot move to any other cell.

Example 1

Input:

N = 4

m[][] =

{{1, 0, 0, 0},

{1, 1, 0, 1},

{1, 1, 0, 0},

{0, 1, 1, 1}}

Output:

DDRDRR DRDDRR

Explanation:

The rat can reach the destination at (3, 3) from (0, 0) by two paths - DRDDRR and DDRDRR, when printed in sorted order we get DDRDRR DRDDRR.

Example 2

Input:

N = 2

m[][] =

{{1, 0},

{1, 0}}

Output:

-1

Explanation:

No path exists and the destination cell is blocked.

vtu29414

Input Format

The first line of input contains an integer n, representing the size of the maze.

The next n lines each contain n space-separated integers, representing the maze.

Output Format

The output consists of

If at least one path is found, print all valid paths as space-separated strings.

If no path is found, prints -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
1 0 0 0
1 1 0 1
1 1 0 0
0 1 1 1
Output: DDRDRR DRDDRR

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int n;
int maze[10][10];
int vis[10][10];
char path[100];
int found = 0;

int dx[] = {1, 0, 0, -1};   // D L R U order for lexicographic output
int dy[] = {0, -1, 1, 0};
char move[] = {'D', 'L', 'R', 'U'};

void dfs(int x, int y, int idx) {
```

```
    if (x == n - 1 && y == n - 1) {
        path[idx] = '\0';
        printf("%s ", path);
        found = 1;
        return;
    }

    vis[x][y] = 1;

    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i];
        int ny = y + dy[i];

        if (nx >= 0 && ny >= 0 && nx < n && ny < n &&
            maze[nx][ny] == 1 && !vis[nx][ny]) {

            path[idx] = move[i];
            dfs(nx, ny, idx + 1);
        }
    }

    vis[x][y] = 0;
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &maze[i][j]);

    if (maze[0][0] == 0 || maze[n-1][n-1] == 0) {
        printf("-1");
        return 0;
    }

    dfs(0, 0, 0);

    if (!found)
        printf("-1");

    return 0;
```

}

Status : Correct ***Marks : 10/10***

3. Problem Statement

Given a partially filled 9x9 2D array, the goal is to assign digits (from 1 to 9) to the empty cells so that every row, column, and sub-grid of size 3x3 contains exactly one instance of the digits from 1 to 9.

Write a program for the same.

Example

Input:

3 0 6 5 0 8 4 0 0

5 2 0 0 0 0 0 0 0

0 8 7 0 0 0 0 3 1

0 0 3 0 1 0 0 8 0

9 0 0 8 6 3 0 0 5

0 5 0 0 9 0 6 0 0

1 3 0 0 0 0 2 5 0

0 0 0 0 0 0 0 7 4

0 0 5 2 0 6 3 0 0

Output:

3 1 6 5 7 8 4 9 2

5 2 9 1 3 4 7 6 8

4 8 7 6 2 9 5 3 1

2 6 3 4 1 5 9 8 7

9 7 4 8 6 3 1 2 5

8 5 1 7 9 2 6 4 3

1 3 8 9 4 7 2 5 6

6 9 2 3 5 1 8 7 4

7 4 5 2 8 6 3 1 9

Explanation

To solve the given Sudoku puzzle, we apply the fundamental rules of Sudoku: each row, column, and 3x3 subgrid must contain the digits 1 through 9 without any repetition. Starting with the given partially filled grid, we systematically fill in the empty cells. For each empty cell, we check the numbers already present in the corresponding row, column, and 3x3 subgrid to determine which number can legally go in that position. This process involves logical deduction and elimination. In the case of the provided puzzle, the empty cells are filled one by one, ensuring that each row, column, and subgrid adheres to the Sudoku rules. For example, in Row 1, the missing numbers (1, 7, 9, 2) are placed in the available spots based on their placement in the columns and subgrids. This continues for each row, progressively narrowing down the possibilities for the empty cells until the entire grid is complete. The final solution, after applying this reasoning, looks like the output grid:

Input Format

The input consists of a 9x9 matrix in 9 rows and 9 columns separated by space.

Output Format

The output displays the 9x9 matrix with the updated values.

If the conditions are not satisfied, then display "No Solution exists".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 0 6 5 0 8 4 0 0
5 2 0 0 0 0 0 0 0
0 8 7 0 0 0 0 3 1

vtu29414

0 0 3 0 1 0 0 8 0
9 0 0 8 6 3 0 0 5
0 5 0 0 9 0 6 0 0
1 3 0 0 0 0 2 5 0
0 0 0 0 0 0 0 7 4
0 0 5 2 0 6 3 0 0

Output: 3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

Answer

```
// You are using GCC
#include <stdio.h>

int grid[9][9];

int isSafe(int r, int c, int num) {
    // row check
    for (int j = 0; j < 9; j++)
        if (grid[r][j] == num) return 0;

    // column check
    for (int i = 0; i < 9; i++)
        if (grid[i][c] == num) return 0;

    // 3x3 box check
    int sr = (r / 3) * 3;
    int sc = (c / 3) * 3;

    for (int i = sr; i < sr + 3; i++)
        for (int j = sc; j < sc + 3; j++)
            if (grid[i][j] == num) return 0;

    return 1;
}
```

```
int solve() {
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {

            if (grid[r][c] == 0) {
                for (int num = 1; num <= 9; num++) {
                    if (isSafe(r, c, num)) {
                        grid[r][c] = num;

                        if (solve())
                            return 1;

                        grid[r][c] = 0; // backtrack
                    }
                }
                return 0; // no number fits
            }
        }
    }
    return 1; // solved
}

int main() {

    for (int i = 0; i < 9; i++)
        for (int j = 0; j < 9; j++)
            scanf("%d", &grid[i][j]);

    if (solve()) {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                printf("%d ", grid[i][j]);
            }
            printf(" ");
        }
    } else {
        printf("No Solution exists");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given an undirected graph and a number m, determine if the graph can be colored with at most m colors such that no two adjacent vertices of the graph are colored with the same color.

Note: Here, the coloring of a graph means the assignment of colors to all vertices.

Example

Input:

0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

3

Output:

Solution Exists:

1 2 3 2

Explanation:

A minimum of 3 colors is required for the above graph.

Input Format

The input consists of four lines each containing four space-separated integers (1 if connected, 0 if not), representing the adjacency matrix of a graph where each element represents the connection between vertices.

The last line consists of an integer m, representing the number of colors available.

Output Format

If a solution exists:

1. The first line prints "Solution Exists:".
2. The second line prints the assigned colors for each vertex, separated by a space.

If no solution exists, the output prints "Solution does not exist".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0
3
Output: Solution Exists:
1 2 3 2

Answer

```
// You are using GCC
#include <stdio.h>

#define V 4

int graph[V][V];
int color[V];
int m;

int isSafe(int v, int c) {
    for (int i = 0; i < V; i++) {
        if (graph[v][i] == 1 && color[i] == c) {
            return 0;
        }
```

```
    }
    return 1;
}

int solve(int v) {
    if (v == V) return 1;

    for (int c = 1; c <= m; c++) {
        if (isSafe(v, c)) {
            color[v] = c;

            if (solve(v + 1)) return 1;

            color[v] = 0;
        }
    }
    return 0;
}

int main() {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    scanf("%d", &m);

    for (int i = 0; i < V; i++) color[i] = 0;

    if (solve(0)) {
        printf("Solution Exists: ");
        for (int i = 0; i < V; i++) {
            printf("%d ", color[i]);
        }
    } else {
        printf("Solution does not exist");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 9_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an array of integers and a sum B, find all unique combinations in the array where the sum is equal to B. The same number may be chosen from the array any number of times to make B.

1. All numbers will be positive integers.

2. Elements in a combination (a1, a2, …, ak) must be in non-descending order. (ie, a1 ≤ a2 ≤ … ≤ ak).

3. The combinations themselves must be sorted in ascending order.

Examples:

Input:

vtu29414

arr[] = {7,2,6,5}

B = 16

Output:

(2 2 2 2 2 2 2 2)

(2 2 2 2 2 6)

(2 2 2 5 5)

(2 2 5 7)

(2 2 6 6)

(2 7 7)

(5 5 6)

Input:

arr[] = {6,5,7,1,8,2,9,9,7,7,9}

B = 6

Output:

(1 1 1 1 1 1)

(1 1 1 1 2)

(1 1 2 2)

(1 5)

(2 2 2)

(6)

Input Format

The first line of input consists of space-separated positive integers representing the elements of the array.

The second line consists of a single positive integer representing the target sum B.

Output Format

The output prints "Combinations that sum to [B] are:" followed by the combinations in parentheses, each combination separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7 2 6 5
16

Output: Combinations that sum to 16 are:
(2 2 2 2 2 2 2 2) (2 2 2 2 2 6) (2 2 2 5 5) (2 2 5 7) (2 2 6 6) (2 7 7) (5 5 6)

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

int arr[100], temp[100], n, B;
int found = 0;

void printCombo(int len) {
    printf("( ");
    for (int i = 0; i < len; i++) {
        printf("%d ", temp[i]);
    }
    printf(")");
}

void dfs(int start, int sum, int len) {
    if (sum == B) {
        printCombo(len);
        printf(" ");
        found = 1;
        return;
    }

    for (int i = start; i < n; i++) {
        if (sum + arr[i] <= B) {
            temp[len] = arr[i];
            dfs(i, sum + arr[i], len);  // allow reuse of same element
```

```
        }
    }
}

int cmp(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int main() {
    char line[500];

    // Read entire first line (array input + B)
    fgets(line, sizeof(line), stdin);

    int nums[100], count = 0;
    char *token = strtok(line, " \n");

    while (token != NULL) {
        nums[count++] = atoi(token);
        token = strtok(NULL, " \n");
    }

    B = nums[count - 1];
    n = count - 1;

    for (int i = 0; i < n; i++) {
        arr[i] = nums[i];
    }

    // sort array
    qsort(arr, n, sizeof(int), cmp);

    printf("Combinations that sum to %d are: ", B);

    dfs(0, 0, 0);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

2. Problem Statement

Imagine you're helping a friend plan gift packages with a total budget of N. Your task is to find all unique ways to spend the entire budget on different combinations of gift prices, ensuring no combination is repeated by rearranging the prices.

Note:

By unique, it means that no matter that no other composition can be expressed as a permutation of the generated composition. For eg. [1,2,1] and [1, 1, 2] are not unique.You need to print all combinations in non-decreasing order, for eg. [1, 2, 1] or [1,1,2] will be printed as [1,1,2].The order of printing all the combinations should be in lexicographical order.

Use backtracking to solve the given problem.

Input Format

The input consists of a single integer, n, which represents the number to be broken down.

Output Format

The output consists of a list of lists, where each inner list represents a possible breakdown of the given number.

Each breakdown is represented as a list of integers that add up to the given number.

The breakdowns are printed in lexicographic order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
Output: [[1]]

Answer

// You are using GCC

vtu29414

```
#include <stdio.h>

int n;
int path[30];
int result[1000][30];
int sizes[1000];
int count = 0;

void dfs(int rem, int start, int len) {
    if (rem == 0) {
        for (int i = 0; i < len; i++) {
            result[count][i] = path[i];
        }
        sizes[count] = len;
        count++;
        return;
    }

    for (int i = start; i <= rem; i++) {
        path[len] = i;
        dfs(rem - i, i, len + 1);  // ensures non-decreasing order
    }
}

int main() {
    scanf("%d", &n);

    dfs(n, 1, 0);

    printf("[");
    for (int i = 0; i < count; i++) {
        printf("[");
        for (int j = 0; j < sizes[i]; j++) {
            printf("%d", result[i][j]);
            if (j < sizes[i] - 1) printf(", ");
        }
        printf("]");
        if (i < count - 1) printf(", ");
    }
    printf("]");

    return 0;
```

}

Status : Correct ***Marks : 10/10***

3. Problem Statement

Rohan is designing a security system that generates all possible rearrangements of a given access code made up of distinct characters. He inputs a string representing the access code into the system. The system must display every possible permutation of the characters without repetition.

Write a program to ensure that all permutations of the given string are printed in separate lines.

Input Format

The input consists of a single line containing a string S, consisting of lowercase or upper-case alphabets.

Output Format

The output prints each permutation of the string on a new line.

The permutations should be printed in the order generated by swapping characters starting from the first position

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: ABC

Output: ABC
ACB
BAC
BCA
CBA
CAB

Answer

```c
#include <stdio.h>
#include <string.h>

void swap(char *a, char *b) {
    char t = *a;
    *a = *b;
    *b = t;
}

void permute(char s[], int l, int r) {
    if (l == r) {
        printf("%s ", s);
        return;
    }

    for (int i = l; i <= r; i++) {
        swap(&s[l], &s[i]);
        permute(s, l + 1, r);
        swap(&s[l], &s[i]); // backtrack
    }
}

int main() {
    char s[10];
    scanf("%s", s);

    int n = strlen(s);
    permute(s, 0, n - 1);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Given a number K and string str of digits denoting a positive integer, build the largest number possible by performing swap operations on the digits of str at most K times.

Input Format

The first line of input consists of a positive integer K.

The second line of input consists of a string of digits of positive integers.

Output Format

The output displays the largest number possible by performing swap operations on the digits of str at most K times.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1
254
Output: 524

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

void swap(char *a, char *b) {
   char t = *a;
   *a = *b;
   *b = t;
}

void findMax(char str[], int k, int n, char maxStr[]) {
   if (k == 0)
      return;

   for (int i = 0; i < n - 1; i++) {
      for (int j = i + 1; j < n; j++) {

         if (str[j] > str[i]) {
            swap(&str[i], &str[j]);

            if (strcmp(str, maxStr) > 0)
               strcpy(maxStr, str);
```

```
            findMax(str, k - 1, n, maxStr);

            swap(&str[i], &str[j]); // backtrack
        }
    }
  }
}

int main() {
  int k;
  char str[35];

  scanf("%d", &k);
  scanf("%s", str);

  char maxStr[35];
  strcpy(maxStr, str);

  int n = strlen(str);

  findMax(str, k, n, maxStr);

  printf("%s", maxStr);

  return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Mythili has a list of item prices and a customer's gift card balance. She needs to find a combination of two or more items whose total price exactly matches the gift card balance. If such a combination exists, she should print the item prices in the order they appear. If multiple valid combinations exist, she must print the first one found. If no valid combination is possible, print "No combination of items"

Write a function that takes the list of item prices, the number of items, and the gift card balance as input, and prints the result based on the above

rules.

Input Format

The first line of input consists of an integer, representing the number of items in the store.

The second line consists of an array of integers separated by a space, representing the prices of the items.

The last line consists of an integer representing the customer's gift card balance.

Output Format

If there exists a combination of items whose prices sum up to the gift card balance, then print this combination.

If no such combination exists, print "No combination of items".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
50
Output: 20 30

Answer

```
// You are using GCC
#include <stdio.h>

int arr[10], n, target;
int sol[10];
int found = 0;

void dfs(int idx, int sum, int len) {
   if (found) return;

   if (sum == target && len >= 2) {
       for (int i = 0; i < len; i++) {
```

```
            printf("%d ", sol[i]);
        }
        found = 1;
        return;
    }

    if (idx == n || sum > target) return;

    // include current element
    sol[len] = arr[idx];
    dfs(idx + 1, sum + arr[idx], len + 1);

    // exclude current element
    dfs(idx + 1, sum, len);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &target);

    dfs(0, 0, 0);

    if (!found) {
        printf("No combination of items");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 9_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 11

Section 1 : MCQ

1. In general, backtracking can be used to solve?

Answer

Combinatorial problems

Status : Correct ***Marks : 1/1***

2. Backtracking may lead to a solution that is _____.

Answer

suboptimal

Status : Correct ***Marks : 1/1***

vtu29414

3. What is the main idea behind Backtracking to check if a graph is connected?

Answer

Trying all possible paths from a starting vertex and marking visited vertices

Status : Correct ***Marks : 1/1***

4. In Backtracking, what is the purpose of marking vertices as visited during graph traversal?

Answer

To avoid revisiting the same vertex

Status : Correct ***Marks : 1/1***

5. In Backtracking, what is the base case for checking connectivity in an undirected graph?

Answer

The graph is a complete graph

Status : Wrong ***Marks : 0/1***

6. In a Sudoku solver, when does the backtracking algorithm backtrack?

Answer

When no valid number can be placed in a cell

Status : Correct ***Marks : 1/1***

7. Which of the following problems cannot be solved using backtracking?

Answer

Finding shortest path in weighted graph

Status : Wrong ***Marks : 0/1***

vtu29414

8. Which of the following is essential for backtracking to function correctly?

Answer

A recursive call with a rollback (backtrack step)

Status : Correct ***Marks : 1/1***

9. What is a base case in a backtracking algorithm?

Answer

A condition when a solution is found or invalid

Status : Correct ***Marks : 1/1***

10. Which data structure is commonly used in backtracking to store partial solutions?

Answer

Stack or List

Status : Correct ***Marks : 1/1***

11. In the Hamiltonian Cycle problem using backtracking, what must be true for a path to be valid?

Answer

All vertices are included once, and the last connects to the first

Status : Correct ***Marks : 1/1***

12. What is the primary technique used to solve the Rat in a Maze problem?

Answer

Dynamic Programming

vtu29414

Status : Wrong ***Marks : 0/1***

13. Which of the following best describes the backtracking approach to generating permutations of a string?

Answer

Fix one character, recursively permute the rest, and backtrack

Status : Correct ***Marks : 1/1***

14. In backtracking solutions for maze problems, when do we backtrack from a cell?

Answer

When all directions from the current cell are either visited or blocked

Status : Correct ***Marks : 1/1***

15. Which of the following constraints are typically used in Sudoku backtracking?

Answer

Column must contain unique digits

Status : Wrong ***Marks : 0/1***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 8_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an M × N matrix of characters, find the length of the longest path in the matrix starting from a given character.

In alphabetical order, all characters in the longest path should be increasing and consecutive. We are allowed to search the string in all eight possible directions, i.e., North, West, South, East, North-East, North-West, South-East, and South-West.

Use depth-first search (DFS) to solve this problem.

For example, consider the following matrix of characters:

The length of the longest path with consecutive characters starting from

character C is 6.

The longest path is:

Input Format

The first line of input is an integer n representing the row size.

The second line of input is an integer m representing the column size.

The following lines of input are the characters of the matrix.

The last line of the input is the character for which the length of the longest path is to be found.

Output Format

The output prints "The length of the longest path with consecutive characters starting from character <startChar> is <length>"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5
d e h x b
a o g p e
d d c f d
e b e a s
c d y e n
c
Output: The length of the longest path with consecutive characters starting from character c is 6

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
#define MAX 5

int n, m;
char mat[MAX][MAX];
int dp[MAX][MAX];

int dx[8] = {-1,-1,-1, 0,0, 1,1,1};
int dy[8] = {-1, 0, 1,-1,1,-1,0,1};

int dfs(int x, int y) {
    if (dp[x][y] != -1)
        return dp[x][y];

    int maxLen = 1;

    for (int dir = 0; dir < 8; dir++) {
        int nx = x + dx[dir];
        int ny = y + dy[dir];

        if (nx >= 0 && ny >= 0 && nx < n && ny < m &&
            mat[nx][ny] == mat[x][y] + 1) {

            int len = 1 + dfs(nx, ny);
            if (len > maxLen)
                maxLen = len;
        }
    }

    return dp[x][y] = maxLen;
}

int main() {
    scanf("%d %d", &n, &m);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf(" %c", &mat[i][j]);
        }
    }

    char start;
    scanf(" %c", &start);
```

```
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            dp[i][j] = -1;

    int ans = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (mat[i][j] == start) {
                int len = dfs(i, j);
                if (len > ans)
                    ans = len;
            }
        }
    }

    printf("The length of the longest path with consecutive characters starting
from character %c is %d", start, ans);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Priya is developing a learning module to simulate breadth-first traversal of directed graphs. Given multiple test cases where each describes a directed graph with vertices and edges, write a program to perform BFS starting from vertex 0 and print the traversal order for each test case.

Example

Input:

1

5 4

0 1 0 2 0 3 2 4

Output:
0 1 2 3 4

Explanation:

For the given graph, the BFS is 0 1 2 3 4.

Input Format

The first line of input contains an integer T, denoting the number of test cases.

Then T test cases follow. Each test case consists of two lines.

The descriptions of the test cases are as follows:

1. The first line of each test case contains two space-separated integers, N and E, which denote the number of vertices and edges, respectively.
2. The second line of each test case contains E space-separated pairs u and v, denoting that there is an edge from u to v.

Output Format

For each test case, the output prints the BFS order of the graph vertices.

Each vertex should appear once in the order it is visited, separated by a space.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 2
5 4
0 1 0 2 0 3 2 4
3 2
0 1 0 2
Output: 0 1 2 3 4
0 1 2

Answer

vtu29414

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int adj[MAX][MAX];
int adjSize[MAX];
int visited[MAX];
int queue[MAX];

void sortAdj(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < adjSize[i] - 1; j++) {
            for (int k = j + 1; k < adjSize[i]; k++) {
                if (adj[i][j] > adj[i][k]) {
                    int temp = adj[i][j];
                    adj[i][j] = adj[i][k];
                    adj[i][k] = temp;
                }
            }
        }
    }
}

void bfs(int start, int n) {
    int front = 0, rear = 0;

    queue[rear++] = start;
    visited[start] = 1;

    while (front < rear) {
        int u = queue[front++];
        printf("%d ", u);

        for (int i = 0; i < adjSize[u]; i++) {
            int v = adj[u][i];
            if (!visited[v]) {
                visited[v] = 1;
                queue[rear++] = v;
            }
        }
```

```
    }
}

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int N, E;
        scanf("%d %d", &N, &E);

        for (int i = 0; i < N; i++) {
            adjSize[i] = 0;
            visited[i] = 0;
        }

        for (int i = 0; i < E; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            adj[u][adjSize[u]++] = v;
        }

        sortAdj(N);

        bfs(0, N);
        printf("\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Aayush is building a network verifier that must detect the presence of cycles in an undirected graph. Given a graph with a set number of vertices and edges, write a program to determine whether the graph contains at least one cycle using depth-first traversal.

Example

Input

N = 4, E = 4

Output

Yes

Explanation

The diagram clearly shows a cycle 0 to 2 to 1 to 0

Input

N = 4, E = 3

Output

No

Explanation

There is no cycle in the given graph.

Input Format

The first line of input consists of two integers, V and E, representing the number of vertices and edges separated by space, respectively.

The next E lines contain two space-separated integers u and v, describing an edge between vertices u and v.

Output Format

The output prints "Graph contains a cycle" if a cycle exists, otherwise prints "Graph does not contain a cycle".

Refer to the sample output for the formatting specifications.

Sample Test Case

vtu29414

Input: 5 5
1 0
0 2
2 1
0 3
3 4

Output: Graph contains a cycle

Answer

```
// You are using GCC
#include <stdio.h>

#define MAX 10

int adj[MAX][MAX];
int adjSize[MAX];
int visited[MAX];

int dfs(int u, int parent) {
    visited[u] = 1;

    for (int i = 0; i < adjSize[u]; i++) {
        int v = adj[u][i];

        if (!visited[v]) {
            if (dfs(v, u))
                return 1;
        }
        else if (v != parent) {
            return 1;
        }
    }

    return 0;
}

int main() {
    int V, E;
    scanf("%d %d", &V, &E);

    for (int i = 0; i < V; i++) {
        adjSize[i] = 0;
```

```
    visited[i] = 0;
    }

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);

        adj[u][adjSize[u]++] = v;
        adj[v][adjSize[v]++] = u;
    }

    int cycle = 0;

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            if (dfs(i, -1)) {
                cycle = 1;
                break;
            }
        }
    }

    if (cycle)
        printf("Graph contains a cycle");
    else
        printf("Graph does not contain a cycle");

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Emma is developing a graph traversal feature for a social networking platform. Each user is represented by a lowercase English letter (a to z), and connections between users are represented as directed edges.

Emma wants to perform a Breadth-First Search (BFS) starting from a given user, visiting friends in lexicographical order whenever multiple

connections are available.

Write a program to help Emma implement this lexicographically sorted BFS traversal.

Input:

N = 10, M = 10, S = ‘a’,

edges[][2] = { { ‘a’, ‘y’ }, { ‘a’, ‘z’ }, { ‘a’, ‘p’ }, { ‘p’, ‘c’ }, { ‘p’, ‘b’ }, { ‘y’, ‘m’ }, { ‘y’, ‘l’ }, { ‘z’, ‘h’ }, { ‘z’, ‘g’ }, { ‘z’, ‘i’ } }

Output:

a p y z b c l m g h i

Input Format

The first line contains an integer N, representing the number of nodes (users) in the graph.

The second line contains an integer M, representing the number of edges (friendships) in the graph.

The next M lines each contain two space-separated lowercase characters u and v, indicating a directed edge from user u to user v.

The last line contains an integer S, the starting node (user) for the BFS traversal.

Output Format

The output prints the users in the order they are visited during the BFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
10
a y

a z
a p
p c
p b
y m
y l
z h
z g
z i
a

Output: a p y z b c l m g h i

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 26

int adj[MAX][MAX];
int adjSize[MAX];
int visited[MAX];
int queue[MAX];

int idx(char c) {
    return c - 'a';
}

char ch(int i) {
    return i + 'a';
}

void sortAdj(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < adjSize[i] - 1; j++) {
            for (int k = j + 1; k < adjSize[i]; k++) {
                if (adj[i][j] > adj[i][k]) {
                    int temp = adj[i][j];
                    adj[i][j] = adj[i][k];
                    adj[i][k] = temp;
                }
            }
```

```
        }
    }
}

void bfs(int start) {
    int front = 0, rear = 0;

    queue[rear++] = start;
    visited[start] = 1;

    while (front < rear) {
        int u = queue[front++];

        printf("%c ", ch(u));

        for (int i = 0; i < adjSize[u]; i++) {
            int v = adj[u][i];
            if (!visited[v]) {
                visited[v] = 1;
                queue[rear++] = v;
            }
        }
    }
}

int main() {
    int N, M;
    scanf("%d %d", &N, &M);

    for (int i = 0; i < MAX; i++) {
        adjSize[i] = 0;
        visited[i] = 0;
    }

    for (int i = 0; i < M; i++) {
        char u, v;
        scanf(" %c %c", &u, &v);

        int a = idx(u);
        int b = idx(v);

        adj[a][adjSize[a]++] = b;
```

```
    }
    char S;
    scanf(" %c", &S);

    sortAdj(MAX);

    bfs(idx(S));

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 8_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Sanjay is writing a program to manage an undirected graph using an adjacency list representation. Users will initially input the vertices to create the edges. The program will display the adjacency list of the graph.

Assist Sanjay in creating the program.

Input Format

The input consists of 5 nodes.

Edge information: ab ae bc bd be cd de.

Output Format

The output prints the adjacency list representation of the graph.

vtu29414

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 0 1 2 3 4

Output: vertex 0: head -> 4 -> 1
vertex 1: head -> 4 -> 3 -> 2 -> 0
vertex 2: head -> 3 -> 1
vertex 3: head -> 4 -> 2 -> 1
vertex 4: head -> 3 -> 1 -> 0

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->next = NULL;
    return node;
}

void addEdge(struct Node* adj[], int u, int v) {
    struct Node* newNode = createNode(v);
    newNode->next = adj[u];
    adj[u] = newNode;

    newNode = createNode(u);
    newNode->next = adj[v];
    adj[v] = newNode;
}

void printGraph(struct Node* adj[], int V) {
    for (int i = 0; i < V; i++) {
```

```
        printf("vertex %d: head", i);
        struct Node* temp = adj[i];

        while (temp != NULL) {
            printf(" -> %d", temp->data);
            temp = temp->next;
        }
        printf(" ");
    }
}

int main() {
    int V = 5;
    int arr[5];

    for (int i = 0; i < V; i++) {
        scanf("%d", &arr[i]);
    }

    struct Node* adj[5];

    for (int i = 0; i < V; i++)
        adj[i] = NULL;

    /* edges: ab ae bc bd be cd de (as per mapping in sample)
       we interpret given input as node order mapping already 0-4 */

    addEdge(adj, 0, 1);
    addEdge(adj, 0, 4);
    addEdge(adj, 1, 2);
    addEdge(adj, 1, 3);
    addEdge(adj, 1, 4);
    addEdge(adj, 2, 3);
    addEdge(adj, 3, 4);

    printGraph(adj, V);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

2. Problem Statement

Sophia is designing a transportation network for a city with multiple interconnected districts. Each district is represented as a vertex, and every road connecting two districts is represented as a directed edge in a graph.

To evaluate connectivity, Sophia needs a program that performs a Breadth-First Search (BFS) traversal of the city's network starting from a specified source district, and outputs the order in which districts are visited.

Write a program to ensure that Sophia can perform the BFS traversal correctly and view the order of visited districts.

Input Format

The first line of input consists of two space-separated integers, v, and e, representing the total number of cities and the number of connections between cities.

The next e lines each contain two space-separated integers a and b, representing a directed edge from city a to city b.

The last line consists of a single integer s, representing the source city from which BFS traversal should start.

Output Format

The output prints the order in which the cities are visited during the BFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6 7
0 1
0 2
1 3
1 4
2 4

vtu29414

3 5
4 5
0
Output: 0 1 2 3 4 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->next = NULL;
    return node;
}

void addEdge(struct Node* adj[], int u, int v) {
    struct Node* node = createNode(v);
    node->next = adj[u];
    adj[u] = node;
}

void bfs(struct Node* adj[], int start, int v) {
    int visited[MAX] = {0};
    int queue[MAX];
    int front = 0, rear = 0;

    queue[rear++] = start;
    visited[start] = 1;

    while (front < rear) {
        int curr = queue[front++];
        printf("%d ", curr);
```

```
        struct Node* temp = adj[curr];
        while (temp != NULL) {
            int nbr = temp->data;
            if (!visited[nbr]) {
                visited[nbr] = 1;
                queue[rear++] = nbr;
            }
            temp = temp->next;
        }
    }
}

int main() {
    int v, e;
    scanf("%d %d", &v, &e);

    struct Node* adj[MAX] = {NULL};

    for (int i = 0; i < e; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        addEdge(adj, a, b);
    }

    int s;
    scanf("%d", &s);

    bfs(adj, s, v);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

3. Problem Statement

Liam is developing a social media analytics tool for tracking user interactions. He needs to identify the connected components of users in the network by traversing a directed graph. Each user is represented as a vertex, and the connections between users are represented as directed

edges. Liam's task is to implement a Depth-First Search (DFS) algorithm to perform a traversal of the network, starting from a given user's profile.

Can you help Liam by implementing the DFS algorithm to identify all connected users starting from a specified user profile?

Example

Input:

6 6

0 1

0 2

1 2

2 0

2 3

3 3

2

Output:

2 0 1 3

Explanation:

Starting from vertex 2, it visits and marks vertex 2 as visited.

Vertex 2 is printed.

Checking the neighbors of vertex 2: 0 and 3.

Vertex 0 is visited, so DFS is called with vertex 0.

Vertex 0 is visited and printed.

Vertex 1 is visited and printed.

Vertex 3 is visited and printed.

The final output is "2 0 1 3".

vtu29414

Input Format

The first line contains two space-separated integers n and m, representing the total number of users (vertices) and the number of connections (edges) in the network, respectively.

The next m lines contain two space-separated integers u and v, representing a directed connection from user u to user v.

The last line contains a single integer s, representing the ID of the source user profile from which the DFS traversal should start.

Output Format

The output displays a single line containing the user IDs in the order they were visited during the DFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6 6
0 1
0 2
1 2
2 0
2 3
3 3
2
Output: 2 0 1 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
    int data;
    struct Node* next;
```

```
};

struct Node* createNode(int val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->next = NULL;
    return node;
}

void addEdge(struct Node* adj[], int u, int v) {
    struct Node* node = createNode(v);
    node->next = adj[u];
    adj[u] = node;
}

void dfs(struct Node* adj[], int v, int visited[]) {
    visited[v] = 1;
    printf("%d ", v);

    struct Node* temp = adj[v];
    while (temp != NULL) {
        int nbr = temp->data;
        if (!visited[nbr]) {
            dfs(adj, nbr, visited);
        }
        temp = temp->next;
    }
}

int main() {
    int n, m;
    scanf("%d %d", &n, &m);

    struct Node* adj[MAX] = {NULL};

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        addEdge(adj, u, v);
    }

    int s;
```

```
    scanf("%d", &s);

    int visited[MAX] = {0};

    dfs(adj, s, visited);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

4. Problem Statement

Given a graph with V vertices numbered from 0 to V-1 and a list of edges, determine whether the graph is bipartite or not.

Note: A bipartite graph is a type of graph where the set of vertices can be divided into two disjoint sets, say U and V, such that every edge connects a vertex in U to a vertex in V, there are no edges between vertices within the same set.

Example:

Input: V = 4, edges[][]= [[0, 1], [0, 2], [1, 2], [2, 3]]

Output: No, the given graph is not Bipartite

Explanation

The graph is not bipartite because no matter how we try to color the nodes using two colors, there exists a cycle of odd length (like 1–2–0–1), which leads to a situation where two adjacent nodes end up with the same color. This violates the bipartite condition, which requires that no two connected nodes share the same color.

Input: V = 4, edges[][] = [[0, 1], [1, 2], [2, 3]]

Output: Yes, the given graph is Bipartite

Explanation:

The given graph can be colored in two colors so, it is a bipartite graph.

Input Format

The first line contains two space-separated integers V and E, representing the number of vertices and edges respectively.

The next E lines each contain two space-separated integers u and v, indicating an undirected edge between vertices u and v.

Output Format

The output print "Yes, the given graph is Bipartite" if the graph is bipartite.

Otherwise, print "No, the given graph is not Bipartite".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4 3
0 1
1 2
2 3
Output: Yes, the given graph is Bipartite

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

struct Node {
   int data;
   struct Node* next;
};

struct Node* createNode(int val) {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
```

```
    node->data = val;
    node->next = NULL;
    return node;
}

void addEdge(struct Node* adj[], int u, int v) {
    struct Node* node = createNode(v);
    node->next = adj[u];
    adj[u] = node;

    node = createNode(u);
    node->next = adj[v];
    adj[v] = node;
}

int bfsCheck(struct Node* adj[], int V, int start, int color[]) {
    int queue[MAX];
    int front = 0, rear = 0;

    queue[rear++] = start;
    color[start] = 0;

    while (front < rear) {
        int u = queue[front++];

        struct Node* temp = adj[u];
        while (temp != NULL) {
            int v = temp->data;

            if (color[v] == -1) {
                color[v] = 1 - color[u];
                queue[rear++] = v;
            }
            else if (color[v] == color[u]) {
                return 0; // not bipartite
            }

            temp = temp->next;
        }
    }
    return 1;
}
```

```
int isBipartite(struct Node* adj[], int V) {
    int color[MAX];

    for (int i = 0; i < V; i++)
        color[i] = -1;

    for (int i = 0; i < V; i++) {
        if (color[i] == -1) {
            if (!bfsCheck(adj, V, i, color))
                return 0;
        }
    }

    return 1;
}

int main() {
    int V, E;
    scanf("%d %d", &V, &E);

    struct Node* adj[MAX] = {NULL};

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        addEdge(adj, u, v);
    }

    if (isBipartite(adj, V))
        printf("Yes, the given graph is Bipartite");
    else
        printf("No, the given graph is not Bipartite");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

David is working on a project that involves traversing graphs for analyzing network connectivity. He is tasked with performing a Depth-First Search (DFS) traversal on a connected, undirected graph to explore all its vertices starting from a given node.

He needs help writing a program that can process multiple test cases where each test case describes a graph with its vertices and edges, and the program should output the DFS traversal order for each graph.

Can you assist David by writing a program that performs the DFS traversal for each test case?

Example

Input:

2

5 4

0 1 0 2 0 3 2 4

4 3

0 1 1 2 0 3

Output:

0 1 2 4 3

0 1 2 3

Explanation:

Visit all the vertices using a depth-first search algorithm using 0 as a source node.

Input Format

The first line of input contains an integer T denoting the number of test cases.

Each test case consists of two lines.

- The first line of each test case contains two integers N and E which denote the number of vertices and number of edges respectively.

- The second line of each test case contains E space-separated pairs u and v denoting that there is an edge from u to v.

Output Format

For each test case, the output displays the graph vertices in the depth-first traversal order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
5 4
0 1 0 2 0 3 2 4
4 3
0 1 1 2 0 3

Output: 0 1 2 4 3
0 1 2 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 10005

int visited[MAX];
int adj[MAX][MAX];
int adjSize[MAX];
int N, E;

void dfs(int u) {
    visited[u] = 1;
    printf("%d ", u);

    for (int i = 0; i < adjSize[u]; i++) {
        int v = adj[u][i];
        if (!visited[v]) {
            dfs(v);
        }
```

```
    }
}

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        scanf("%d %d", &N, &E);

        for (int i = 0; i < N; i++) {
            adjSize[i] = 0;
            visited[i] = 0;
        }

        for (int i = 0; i < E; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
            adj[u][adjSize[u]++] = v;
            adj[v][adjSize[v]++] = u;  // undirected graph
        }

        dfs(0);
        printf("\n");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 8_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 3

Section 1 : MCQ

1. Which data structure is used in Breadth First Search?

Answer

Linked List

Status : Wrong ***Marks : 0/1***

2. Which of the following problems can be solved by using both BFS & DFS?

Answer

Finding the shortest path between two vertices in an unweighted graph

Status : Correct ***Marks : 1/1***

vtu29414

3. In BFS, what happens when a node is visited for the second time?

Answer

It is skipped and the traversal continues to the next unvisited node

Status : Correct ***Marks : 1/1***

4. Given two vertices in a graph s and t, which of the two traversals (BFS and DFS) can be used to find if there is path from s to t?

Answer

Neither BFS nor DFS

Status : Wrong ***Marks : 0/1***

5. What will be the result of depth-first traversal in the following tree?

Answer

4 5 2 3 1

Status : Wrong ***Marks : 0/1***

6. The Breadth First Search (BFS) algorithm has been implemented using the queue data structure. Which one of the following is a possible order of visiting the nodes in the graph below?

Answer

NQMPOR

Status : Wrong ***Marks : 0/1***

7. In the _____ traversal, we process all of a vertex's descendants before we move to an adjacent vertex.

Answer

vtu29414

Breadth First

Status : Wrong ***Marks : 0/1***

8. Which of the following conditions is sufficient to detect a cycle in a directed graph?

Answer

There is an edge from the currently visited node to an ancestor of the currently visited node in the DFS forest.

Status : Correct ***Marks : 1/1***

9. Regarding the implementation of Breadth First Search using queues, what is the maximum distance between two nodes present in the queue? (considering each edge length 1)

Answer

Insufficient Information

Status : Wrong ***Marks : 0/1***

10. Given the following adjacency matrix of a graph(G) determine the number of components in the G.

[0 1 1 0 0 0],

[1 0 1 0 0 0],

[1 1 0 0 0 0],

[0 0 0 0 1 0],

[0 0 0 1 0 0],

[0 0 0 0 0 0].

Answer

1

Status : Wrong ***Marks : 0/1***

11. Which of the following statements is true about DFS?

Answer

DFS always finds the shortest path between two nodes in a graph.

Status : Wrong ***Marks : 0/1***

12. Consider the following graph.

Apply breadth-first search traversal of the above graph. Which of the following traversal is possible if the start vertex is G? (Assume lexicographic ordering)

Answer

GAHJIBCDEK

Status : Wrong ***Marks : 0/1***

13. Which of the following is a possible result of depth-first traversal of the given graph(consider 1 to be the source element)?

Answer

1 2 3 1 4 5

Status : Wrong ***Marks : 0/1***

14. In an adjacency list representation of a graph, what does each node in the list represent?

Answer

Neither vertices nor edges

Status : Wrong ***Marks : 0/1***

15. The number of elements in the adjacency matrix of a graph having 7

vertices is _____.

Answer

7

Status : Wrong ***Marks : 0/1***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 7_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 37.5

Section 1 : Coding

1. Problem Statement

Given a BST, write an efficient function to delete a given key in it.

There are three cases that can be considered:

Deleting a node with no children: remove the node from the tree.Deleting a node with two childrenDeleting a node with one child

Input Format

The first line of the input consists of an integer n representing, the number of nodes.

The second line of the input consists of integers space-separated representing, input node values.

The third line of the input consists of an integer q representing, the node value to

be deleted.

Output Format

The output prints the node values before and after the deletion of the key value in the format:

"Before deletion of key value:"

"After deleting key value:"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 30 5 6 7
30
Output: Before deletion of key value:
5 6 7 10 30
After deleting key value:
5 6 7 10

Answer

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* createNode(int val) {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = val;
   node->left = node->right = NULL;
   return node;
}

struct Node* insert(struct Node* root, int val) {
   if (root == NULL)
```

```
        return createNode(val);

    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);

    return root;
}

void inorder(struct Node* root) {
    if (root == NULL)
        return;

    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

struct Node* findMin(struct Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL)
        return NULL;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // no child
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        // one child
        else if (root->left == NULL) {
            struct Node* temp = root->right;
```

```
        free(root);
        return temp;
    }
    else if (root->right == NULL) {
        struct Node* temp = root->left;
        free(root);
        return temp;
    }
    // two children
    else {
        struct Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
  }

  return root;
}

int main() {
  int n, q;
  scanf("%d", &n);

  struct Node* root = NULL;
  int val;

  for (int i = 0; i < n; i++) {
    scanf("%d", &val);
    root = insert(root, val);
  }

  scanf("%d", &q);

  printf("Before deletion of key value: ");
  inorder(root);

  root = deleteNode(root, q);

  printf("After deleting key value: ");
  inorder(root);

  return 0;
```

}

Status : Correct ***Marks : 10/10***

2. Problem Statement

Revi is studying binary search trees (BSTs) and wants to explore all the unique BSTs that can be constructed using distinct integers from 1 to n. He decides to write a program that generates all such BSTs and prints their preorder traversals.

Write a program to ensure that the preorder traversal of all possible BSTs constructed from numbers 1 to n is printed, each on a new line.

Example

Input

3

Output

1 2 3

1 3 2

2 1 3

3 1 2

3 2 1

Explanation

Input Format

The input consists of a single integer n, representing the number of nodes.

Output Format

Each line contains the preorder traversal (space-separated) of one possible BST using values from 1 to n.

Each traversal is printed on a new line.

There should be no blank lines between the outputs, and the order of output may vary depending on the internal recursive tree generation.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 1 2 3
1 3 2
2 1 3
3 1 2
3 2 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* createNode(int val) {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = val;
   node->left = node->right = NULL;
   return node;
}

// copy tree structure
struct Node* clone(struct Node* root) {
   if (root == NULL) return NULL;

   struct Node* newNode = createNode(root->data);
   newNode->left = clone(root->left);
   newNode->right = clone(root->right);
```

```
    return newNode;
}

// preorder print
void preorder(struct Node* root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

// generate all BSTs
void generate(int start, int end, struct Node*** list, int* size) {
    if (start > end) {
        list[*size] = NULL;
        (*size)++;
        return;
    }

    for (int i = start; i <= end; i++) {
        struct Node** leftList = (struct Node**)malloc(100 * sizeof(struct Node*));
        struct Node** rightList = (struct Node**)malloc(100 * sizeof(struct Node*));
        int leftSize = 0, rightSize = 0;

        generate(start, i - 1, leftList, &leftSize);
        generate(i + 1, end, rightList, &rightSize);

        for (int l = 0; l < leftSize; l++) {
            for (int r = 0; r < rightSize; r++) {

                struct Node* root = createNode(i);
                root->left = clone(leftList[l]);
                root->right = clone(rightList[r]);

                list[*size] = root;
                (*size)++;
            }
        }
    }
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    struct Node** trees = (struct Node**)malloc(1000 * sizeof(struct Node*));
    int size = 0;

    generate(1, n, trees, &size);

    for (int i = 0; i < size; i++) {
        preorder(trees[i]);
    }

    return 0;
}
```

Status : Partially correct ***Marks : 7.5/10***

3. Problem Statement

Jai is fascinated by binary trees and wants to create and manipulate them. He needs your help to implement basic operations like insertion and deletion in a binary search tree.

Given a sequence of characters representing the values to be inserted into a binary search tree, followed by a character V to be deleted from the tree, your task is to construct the binary search tree and delete the specified character from it.

Your program should perform the following operations:

Construct a binary search tree from the given sequence of characters.Delete the specified character V from the constructed tree.Print the elements of the tree in sorted order after deletion.

Input Format

The first line contains an integer N, denoting the number of characters in the sequence.

The second line contains N space-separated characters, representing the

sequence of characters to be inserted into the binary search tree.

The third line contains a single character V (which is guaranteed to be present in the tree), representing the value to be deleted from the binary search tree.

Output Format

The output prints a single line containing the elements of the binary search tree in sorted order after deleting the specified character V.

If the specified value is not available in the tree, print the given input values in order traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
F B G A D
B

Output: A D F G

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node* createNode(char val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->left = node->right = NULL;
    return node;
}

struct Node* insert(struct Node* root, char val) {
    if (root == NULL)
```

```
        return createNode(val);

    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);

    return root;
}

void inorder(struct Node* root) {
    if (root == NULL) return;

    inorder(root->left);
    printf("%c ", root->data);
    inorder(root->right);
}

struct Node* findMin(struct Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

struct Node* deleteNode(struct Node* root, char key) {
    if (root == NULL)
        return NULL;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        else if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        }
```

```
        else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
        else {
            struct Node* temp = findMin(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data);
        }
    }

    return root;
}

int main() {
    int n;
    scanf("%d", &n);

    struct Node* root = NULL;
    char val, del;

    for (int i = 0; i < n; i++) {
        scanf(" %c", &val);
        root = insert(root, val);
    }

    scanf(" %c", &del);

    root = deleteNode(root, del);

    inorder(root);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement:

Arun is exploring operations on binary search trees (BST). He wants to

write a program that takes an unsorted array of distinct integers and constructs a BST by inserting elements in the given order (not height-balanced).

After constructing the BST, he wants to delete all nodes at the deepest level of the tree and perform a post-order traversal to print the remaining nodes.

Example

Input:

7

1 7 4 9 2 11 12

Output:

Before deleting: 2 4 12 11 9 7 1

After deleting: 2 4 11 9 7 1

Explanation:

The BST is constructed by inserting elements in the given order: 1, 7, 4, 9, 2, 11, 12. This creates a regular BST (not height-balanced) where each element is inserted according to BST properties.

The tree is traversed in post-order before deleting the deepest level, showing: 2, 4, 12, 11, 9, 7, 1.

The deepest level nodes (in this case, node 12) are then deleted.

The tree is traversed in post-order after deleting the deepest level, showing: 2, 4, 11, 9, 7, 1.

Input Format

The first line contains an integer n, representing the number of nodes to be inserted in the BST.

The second line contains n space-separated integers, representing the elements

to be inserted into the BST in the given order.

Output Format

The first line prints "Before deleting: " followed by the space-separated elements of the tree in post-order traversal before deleting the deepest level, ending with a space and newline.

The second line prints "After deleting: " followed by the space-separated elements of the tree in post-order traversal after deleting the deepest level, ending with a space and newline.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
1 7 4 9 2 11 12

Output: Before deleting: 2 4 12 11 9 7 1
After deleting: 2 4 11 9 7 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* createNode(int val) {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = val;
   node->left = node->right = NULL;
   return node;
}

struct Node* insert(struct Node* root, int val) {
   if (root == NULL)
      return createNode(val);
```

```
    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);

    return root;
}

void postorder(struct Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

int maxDepth(struct Node* root) {
    if (root == NULL) return 0;

    int l = maxDepth(root->left);
    int r = maxDepth(root->right);

    return (l > r ? l : r) + 1;
}

struct Node* deleteDeepest(struct Node* root, int depth, int cur) {
    if (root == NULL) return NULL;

    if (cur == depth && root->left == NULL && root->right == NULL) {
        free(root);
        return NULL;
    }

    root->left = deleteDeepest(root->left, depth, cur + 1);
    root->right = deleteDeepest(root->right, depth, cur + 1);

    return root;
}

int main() {
    int n;
    scanf("%d", &n);
```

```
    struct Node* root = NULL;
    int arr[25];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        root = insert(root, arr[i]);
    }

    printf("Before deleting: ");
    postorder(root);
    printf("\n");

    int depth = maxDepth(root);

    root = deleteDeepest(root, depth, 1);

    printf("After deleting: ");
    postorder(root);
    printf("\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 7_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 47.5

Section 1 : Coding

1. Problem Statement

A publishing company is building a digital index system for storing book references. Each reference is represented by a single uppercase character. To ensure efficient organization, the system uses a Binary Search Tree (BST):

If the new character is smaller than the current node, it is placed in the left subtree.If the new character is greater, it is placed in the right subtree.

Once all references are stored in the BST, the system needs to perform a postorder traversal to generate a summary sequence, which is then used for indexing and cross-referencing.

Your task is to help the publishing company build the BST from the given characters and print the postorder traversal.

Example:

Input:

5

Z E W T Y

Output:

Postorder traversal: T Y W E Z

Input Format

The first line consists of an integer N, representing the number of characters.

The second line consists of N space-separated characters.

Output Format

The output displays the post-order traversal of the given inputs as space-separated.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
Z E W T Y
Output: Postorder traversal: T Y W E Z

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   char data;
   struct Node *left, *right;
};

struct Node* createNode(char val) {
```

```
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->left = node->right = NULL;
    return node;
}

struct Node* insert(struct Node* root, char val) {
    if (root == NULL)
        return createNode(val);

    if (val < root->data)
        root->left = insert(root->left, val);
    else if (val > root->data)
        root->right = insert(root->right, val);

    return root;
}

void postorder(struct Node* root) {
    if (root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    printf("%c ", root->data);
}

int main() {
    int n;
    scanf("%d", &n);

    struct Node* root = NULL;
    char val;

    for (int i = 0; i < n; i++) {
        scanf(" %c", &val);
        root = insert(root, val);
    }

    printf("Postorder traversal: ");
    postorder(root);
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Thiru is learning about binary search trees and wants to practice creating and traversing them. He needs to write a program that takes a series of characters as input and constructs a binary search tree with those characters.

After constructing the tree, Thiru wants to perform an in-order traversal and print all the characters in sorted order.

Input Format

The first line contains an integer n, representing the number of characters to be inserted into the binary search tree.

The second line contains n space-separated characters c_i, representing the characters to be inserted into the binary search tree.

Output Format

The output prints a single line containing all the characters of the binary search tree in sorted order, obtained by performing an in-order traversal of the tree, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
r e f s
Output: e f r s

Answer

// You are using GCC

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node* createNode(char val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->left = node->right = NULL;
    return node;
}

struct Node* insert(struct Node* root, char val) {
    if (root == NULL)
        return createNode(val);

    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);

    return root;
}

void inorder(struct Node* root) {
    if (root == NULL)
        return;

    inorder(root->left);
    printf("%c ", root->data);
    inorder(root->right);
}

int main() {
    int n;
    scanf("%d", &n);

    struct Node* root = NULL;
    char val;
```

```
    for (int i = 0; i < n; i++) {
        scanf(" %c", &val);
        root = insert(root, val);
    }

    inorder(root);

    return 0;
}
```

Status : Partially correct ***Marks : 7.5/10***

3. Problem Statement

Thilak is studying binary search trees (BSTs) and wants to implement a program that constructs a binary search tree from a given set of characters. He aims to perform a pre-order traversal of the constructed tree to understand its structure better.

Your task is to help Thilak by designing a program that accomplishes this.

Input Format

The first line of input consists of an integer N, representing the number of characters to be inserted into the BST.

The second line consists of N space-separated characters, representing the data elements to be inserted into the BST.

Output Format

The output prints the pre-order traversal of the constructed BST.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
C A E B D

Output: C A B E D

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node* createNode(char val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->left = node->right = NULL;
    return node;
}

struct Node* insert(struct Node* root, char val) {
    if (root == NULL)
        return createNode(val);

    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);

    return root;
}

void preorder(struct Node* root) {
    if (root == NULL)
        return;

    printf("%c ", root->data);
    preorder(root->left);
    preorder(root->right);
}

int main() {
    int n;
```

```
    scanf("%d", &n);

    struct Node* root = NULL;
    char val;

    for (int i = 0; i < n; i++) {
        scanf(" %c", &val);
        root = insert(root, val);
    }

    preorder(root);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Jake is learning about binary search trees(BST) and their operations. He wants to implement a program that can delete a node from a BST based on the given key value and print the remaining nodes in an in-order traversal.

Assist Jake in the program.

Input Format

The first line of input consists of an integer n, representing the number of elements in BST.

The second line consists of n space-separated integers, representing the elements of the tree.

The third line consists of an integer x, representing the key value of the node to be deleted.

Output Format

The first line of output prints "Before deletion: " followed by the in-order traversal of the initial BST.

The second line prints "After deletion: " followed by the in-order traversal after the deletion of the key value.

If the key value is not present in the BST, print the original tree as it is.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
8 6 4 3 1
4
Output: Before deletion: 1 3 4 6 8
After deletion: 1 3 6 8

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* createNode(int val) {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = val;
   node->left = node->right = NULL;
   return node;
}

struct Node* insert(struct Node* root, int val) {
   if (root == NULL)
      return createNode(val);

   if (val < root->data)
      root->left = insert(root->left, val);
   else
      root->right = insert(root->right, val);
```

```
    return root;
}

void inorder(struct Node* root) {
    if (root == NULL) return;

    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

struct Node* findMin(struct Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

struct Node* deleteNode(struct Node* root, int key, int *found) {
    if (root == NULL)
        return NULL;

    if (key < root->data) {
        root->left = deleteNode(root->left, key, found);
    }
    else if (key > root->data) {
        root->right = deleteNode(root->right, key, found);
    }
    else {
        *found = 1;

        // no child
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        // one child
        else if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        }
```

```
        else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
        // two children
        else {
            struct Node* temp = findMin(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data, found);
        }
    }

    return root;
}

int main() {
    int n, x;
    scanf("%d", &n);

    struct Node* root = NULL;
    int val;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }

    scanf("%d", &x);

    printf("Before deletion: ");
    inorder(root);

    int found = 0;
    root = deleteNode(root, x, &found);

    printf("After deletion: ");
    inorder(root);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

You are required to implement basic operations on a Binary Search Tree (BST), like insertion and searching.

Insertion: Given a list of integers, construct a Binary Search Tree by repeatedly inserting each integer into the tree according to the rules of a BST.

Searching: Given an integer, search for its presence in the constructed Binary Search Tree. Print whether the integer is found or not.

Write a program to calculate this efficiently.

Input Format

The first line of input consists of an integer n, representing the number of nodes in the binary search tree.

The second line consists of the values of the nodes, separated by space as integers.

The third line consists of an integer representing, the value that is to be searched.

Output Format

The output prints, "Value <value> is found in the tree." if the given value is present, otherwise it prints: "Value <value> is not found in the tree."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
8 3 10 1 6 14 23
6
Output: Value 6 is found in the tree.

vtu29414

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* createNode(int val) {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = val;
   node->left = node->right = NULL;
   return node;
}

struct Node* insert(struct Node* root, int val) {
   if (root == NULL)
      return createNode(val);

   if (val < root->data)
      root->left = insert(root->left, val);
   else
      root->right = insert(root->right, val);

   return root;
}

int search(struct Node* root, int key) {
   if (root == NULL)
      return 0;

   if (root->data == key)
      return 1;

   if (key < root->data)
      return search(root->left, key);
   else
      return search(root->right, key);
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    struct Node* root = NULL;
    int val;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }

    int key;
    scanf("%d", &key);

    if (search(root, key)) {
        printf("Value %d is found in the tree.", key);
    } else {
        printf("Value %d is not found in the tree.", key);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 7_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 1

Section 1 : MCQ

1. Which of the following operations can be used to traverse a Binary Search Tree (BST) in ascending order?

Answer

Level-order traversal

Status : Wrong ***Marks : 0/1***

2. Find the pre-order traversal of the given binary search tree.

Answer

18, 14, 13,1, 4, 2

Status : Wrong ***Marks : 0/1***

vtu29414

3. Find the postorder traversal of the given binary search tree.

Answer

18, 14, 13,1, 4, 2

Status : Wrong ***Marks : 0/1***

4. Find the inorder traversal of the given binary search tree.

Answer

2, 14, 10, 9, 1, 4, 7, 6

Status : Wrong ***Marks : 0/1***

5. When comparing a value to be searched with the root node in a BST?

Answer

If it's equal, the search is restart in root

Status : Wrong ***Marks : 0/1***

6. In a binary search tree (BST), if the value to be searched is smaller than the value of the root, where should the search proceed?

Answer

None of the mentioned options

Status : Wrong ***Marks : 0/1***

7. In a binary search tree with nodes 18, 28, 12, 11, 16, 14, 17, what is the value of the right child of the node 12?

Answer

vtu29414

11

Status : Wrong ***Marks : 0/1***

8. In a binary search tree with nodes 18, 28, 12, 11, 16, 14, 17, what is the value of the left child of the node 16?

Answer

11

Status : Wrong ***Marks : 0/1***

9. The preorder traversal sequence of a binary search tree is 30, 20, 10, 15, 25, 23, 39, 35, 42. Which one of the following is the post-order traversal sequence of the same tree?

Answer

15, 10, 25, 23, 20, 42, 35, 39, 30

Status : Wrong ***Marks : 0/1***

10. In a binary search tree, which of the following traversals is used for getting ascending order values?

Answer

Level-order

Status : Wrong ***Marks : 0/1***

11. Consider the given binary search tree, if the root node is deleted, then the new root can be _____.

Answer

43 or 48

Status : Wrong ***Marks : 0/1***

12. After deleting a node in a binary search tree, how is the replacement node determined if the node to be deleted has only one child?

Answer

Replacement node is the child node itself

Status : Correct ***Marks : 1/1***

13. What will be the in-order traversal sequence of the following binary search tree, after deleting node 40?

Answer

12 20 22 30 8

Status : Wrong ***Marks : 0/1***

14. Which of the following nodes would replace node 40 if it were to be deleted?

Answer

4

Status : Wrong ***Marks : 0/1***

15. Which of the following statements is true about deleting a node with one child from a binary search tree?

Answer

The node is directly removed, and its child is merged with grand parent

Status : Wrong ***Marks : 0/1***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 6_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Yuvana, an avid programmer, is keen on mastering various sorting algorithms and has recently delved into Counting Sort. As a practice exercise, she is given an array of N integers where each element ranges from 1 to N-1. Some numbers may appear multiple times in the array. Her task is to identify and print all numbers that appear more than once. If no such duplicates are found, she should print -1.

Help Yuvana by implementing a function that efficiently finds and prints duplicate numbers using the Counting Sort technique.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

The output prints all the duplicate numbers that appear more than once in the array, each separated by a space.

If no duplicates are found, print -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 3 4 2 2

Output: 2

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];
    int count[20] = {0};

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        count[arr[i]]++;
    }

    int found = 0;

    for (int i = 1; i < n; i++) {
        if (count[i] > 1) {
            printf("%d ", i);
            found = 1;
```

```
        }
    }

    if (!found) {
        printf("-1");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Anu is tasked with organizing a list of fruit names stored as strings. To ensure that the list is easy to navigate, she decides to sort the strings in ascending alphabetical order.

Being a fan of efficient sorting algorithms, Anu opts to use radix sort for this task. Help Anu with a program that can take a list of fruit names as input and output the sorted list.

Input Format

The first line of input consists of an integer n, representing the number of fruit names in the list.

The next n lines each consist of a string, representing the names of the fruits.

Output Format

The output prints n lines containing the sorted list of fruit names in ascending alphabetical order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
apple

banana
orange
grape

Output: apple
banana
grape
orange

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 20
#define LEN 101

int maxLen(char arr[][LEN], int n) {
    int max = strlen(arr[0]);
    for (int i = 1; i < n; i++) {
        int len = strlen(arr[i]);
        if (len > max)
            max = len;
    }
    return max;
}

// counting sort based on character position
void countingSort(char arr[][LEN], int n, int pos) {
    char output[MAX][LEN];
    int count[257] = {0}; // ASCII + 1 (for 0 padding)

    // frequency count
    for (int i = 0; i < n; i++) {
        int index = (pos < strlen(arr[i])) ? (unsigned char)arr[i][strlen(arr[i]) - pos - 1] :
0;
        count[index]++;
    }

    // cumulative count
    for (int i = 1; i < 257; i++) {
        count[i] += count[i - 1];
    }
```

```
    // build output (stable)
    for (int i = n - 1; i >= 0; i--) {
        int index = (pos < strlen(arr[i])) ? (unsigned char)arr[i][strlen(arr[i]) - pos - 1] :
0;
        strcpy(output[count[index] - 1], arr[i]);
        count[index]--;
    }

    // copy back
    for (int i = 0; i < n; i++) {
        strcpy(arr[i], output[i]);
    }
}

void radixSort(char arr[][LEN], int n) {
    int m = maxLen(arr, n);

    for (int pos = 0; pos < m; pos++) {
        countingSort(arr, n, pos);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[MAX][LEN];

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    radixSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%s ", arr[i]);
    }

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

3. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500
5

Output: 5th largest Salary: 28000

Answer

```
// You are using GCC
#include <stdio.h>
```

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// heapify for max heap
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// build heap
void buildHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

int extractMax(int arr[], int *n) {
    int max = arr[0];
    arr[0] = arr[*n - 1];
    (*n)--;
    heapify(arr, *n, 0);
    return max;
}

int main() {
    int n, k;
    scanf("%d", &n);
```

```
    int arr[102];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &k);

    buildHeap(arr, n);

    int size = n;
    int ans = -1;

    for (int i = 0; i < k; i++) {
        ans = extractMax(arr, &size);
    }

    printf("%dth largest Salary: %d ", k, ans);

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Manage a warehouse inventory with unique barcode numbers utilizing a specialized sorting algorithm designed for barcodes with exactly 16 digits. Input the number of barcode numbers (N) and their corresponding values.

Employ the Bucket Sort algorithm to distribute and sort the numbers based on their digits. Display the sorted barcode numbers, ensuring an efficient and organized warehouse inventory management system tailored for barcode systems with 16 digits.

Input Format

The first line contains an integer N, representing the number of barcode numbers to be sorted.

The following N lines contain exactly 16-digit long long integers, each representing a unique barcode number.

Output Format

The output displays a single line containing "Sorted barcode numbers:" followed by the sorted barcode numbers in ascending order. Each barcode number is displayed on a separate line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
2345678901234567
1234567890123456
3456789012345678
5678901234567890
4567890123456789

Output: Sorted barcode numbers:
1234567890123456
2345678901234567
3456789012345678
4567890123456789
5678901234567890

Answer

```
// You are using GCC
#include <stdio.h>

void swap(unsigned long long *a, unsigned long long *b) {
   unsigned long long temp = *a;
   *a = *b;
   *b = temp;
}

void heapify(unsigned long long arr[], int n, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
```

```
    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(unsigned long long arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    unsigned long long arr[100];

    for (int i = 0; i < n; i++) {
        scanf("%llu", &arr[i]);
    }

    heapSort(arr, n);

    printf("Sorted barcode numbers:");

    for (int i = 0; i < n; i++) {
        printf(" %llu", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 6_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Krish is participating in a coding competition and has been given an array of integers. His task is to sort the array using the Counting Sort algorithm.

Your goal is to help Krish by implementing the countSort method to sort the array in non-decreasing order.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

vtu29414

The output prints a single line containing N space-separated integers representing the sorted array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
4 2 2 8 3

Output: 2 2 3 4 8

Answer

```
// You are using GCC
#include <stdio.h>

void countSort(int arr[], int n) {
    int count[101] = {0};

    // store frequency
    for (int i = 0; i < n; i++) {
        count[arr[i]]++;
    }

    // rebuild array
    int index = 0;
    for (int i = 0; i <= 100; i++) {
        while (count[i] > 0) {
            arr[index++] = i;
            count[i]--;
        }
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[15];

    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
    }

    countSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Raj, a keen problem solver, has come across an intriguing numerical puzzle that involves sorting numbers in ascending order. Eager to crack the code, he decides to implement the Radix Sort algorithm.

Your task is to assist Raj in designing a program to solve this numerical puzzle.

Input Format

The first line contains an integer n, denoting the number of elements to be sorted.

The second line consists of n space-separated integers, representing the numbers to be sorted.

Output Format

The output prints the array after it has been sorted using the Radix Sort algorithm, separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
543 986 217 765 329
Output: 217 329 543 765 986

Answer

```
// You are using GCC
#include <stdio.h>

int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max)
            max = arr[i];
    }
    return max;
}

// counting sort based on digit (exp)
void countingSort(int arr[], int n, int exp) {
    int output[20];
    int count[10] = {0};

    // count occurrences
    for (int i = 0; i < n; i++) {
        int digit = (arr[i] / exp) % 10;
        count[digit]++;
    }

    // cumulative count
    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }

    // build output array (stable sort)
    for (int i = n - 1; i >= 0; i--) {
        int digit = (arr[i] / exp) % 10;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    // copy back
    for (int i = 0; i < n; i++) {
```

```
        arr[i] = output[i];
    }
}

// radix sort
void radixSort(int arr[], int n) {
    int max = getMax(arr, n);

    for (int exp = 1; max / exp > 0; exp *= 10) {
        countingSort(arr, n, exp);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    radixSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2
Output: 20

Answer

```
// You are using GCC
#include <stdio.h>

void swap(long long *a, long long *b) {
   long long temp = *a;
   *a = *b;
   *b = temp;
}

// heapify
void heapify(long long arr[], int n, int i) {
   int largest = i;
```

```
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

// heap sort (descending order)
void heapSort(long long arr[], int n) {
    // build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // extract elements
    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int n, k;
    scanf("%d", &n);

    long long arr[100000];

    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    scanf("%d", &k);

    heapSort(arr, n);
```

```
    // Kth best-selling (descending order    index k-1)
    printf("%lld", arr[k - 1]);

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

4. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10
banana pineapple mango chikoo jack star guava lemon tomato orange
Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
// You are using GCC
#include <stdio.h>
```

```
#include <string.h>

#define MAX 20
#define LEN 100

void swap(char a[], char b[]) {
    char temp[LEN];
    strcpy(temp, a);
    strcpy(a, b);
    strcpy(b, temp);
}

// heapify (min heap for lexicographic order)
void heapify(char arr[][LEN], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && strcmp(arr[left], arr[smallest]) < 0)
        smallest = left;

    if (right < n && strcmp(arr[right], arr[smallest]) < 0)
        smallest = right;

    if (smallest != i) {
        swap(arr[i], arr[smallest]);
        heapify(arr, n, smallest);
    }
}

void heapSort(char arr[][LEN], int n) {
    // build heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // extract elements
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    char arr[MAX][LEN];

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    heapSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%s ", arr[i]);
    }

    return 0;
}
```

Status : Wrong ***Marks : 0/10***

5. Problem Statement

You're organizing a gift-giving event where participants exchange gifts of varying values. To ensure fairness, you use the Bucket Sort algorithm. You gather the participants' gift values and initialize a count array.

Then, you iterate through the values, updating the count array. Next, you sort the values in descending order by iterating through the count array. Finally, you display the sorted values, ensuring fairness in the gift exchange.

Input Format

The first line contains an integer N, representing the number of gift values.

The second line contains N space-separated integers representing the gift values brought by the participants. Each gift value is an integer between 1 and 999, inclusive.

Output Format

The output displays a single line containing N space-separated integers

representing the sorted gift values in descending order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
156 378 453 278 281 987
Output: 987 453 378 281 278 156

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[100];
    int count[1000] = {0};

    // read input and build frequency array
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        count[arr[i]]++;
    }

    // print in descending order
    for (int i = 999; i >= 1; i--) {
        while (count[i] > 0) {
            printf("%d ", i);
            count[i]--;
        }
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 6_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 5

Section 1 : MCQ

1. In Radix Sort, what happens if the input array contains numbers with different lengths?

Answer

The numbers are converted to the same length.

Status : Wrong ***Marks : 0/1***

2. In heap sort, when does the actual sorting take place?

Answer

During the heap construction phase

Status : Wrong ***Marks : 0/1***

vtu29414

3. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

None of the mentioned options

Status : Wrong ***Marks : 0/1***

4. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

None of the mentioned options

Status : Wrong ***Marks : 0/1***

5. Bucket sort is most efficient in the case when _____

Answer

The input is non-uniformly distributed

Status : Wrong ***Marks : 0/1***

6. Which of the following is not necessarily a stable sorting algorithm?

Answer

All of the mentioned options

Status : Wrong ***Marks : 0/1***

7. Which step is essential to implement Bucket Sort correctly?

Answer

Dividing the range of input into intervals (buckets)

Status : Correct ***Marks : 1/1***

vtu29414

8. Which of the following is a characteristic of Radix Sort?

Answer

It is a divide-and-conquer sorting algorithm.

Status : Wrong ***Marks : 0/1***

9. A database contains the following employee IDs: [1263, 2910, 8742, 3891, 2156]. Using Radix Sort, which of the following is the correct order of employee IDs after the first pass (sorting by the least significant digit)?

Answer

[1263, 3891, 2156, 2910, 8742]

Status : Wrong ***Marks : 0/1***

10. What is the purpose of the counting sort algorithm in Radix Sort?

Answer

To sort the elements based on the current digit.

Status : Correct ***Marks : 1/1***

11. When does Heap Sort perform at its worst?

Answer

When the input array elements are required to be sorted in reverse order.

Status : Wrong ***Marks : 0/1***

12. How does Counting Sort handle duplicate elements?

Answer

It preserves their relative order

Status : Correct ***Marks : 1/1***

vtu29414

13. What is the purpose of the count array in Counting Sort?

Answer

To store the count of each unique element of the input array at their respective indices

Status : Correct ***Marks : 1/1***

14. What is the purpose of calculating the prefix sum in the Counting Sort algorithm?

Answer

To determine the final positions of elements in the output array

Status : Correct ***Marks : 1/1***

15. What is the initial value of each element in the count array in Counting Sort?

Answer

The value of the input element

Status : Wrong ***Marks : 0/1***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 5_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given a singly linked list and two integers M and N, implement a program to traverse the linked list such that you retain every M node and then delete the next N nodes.

Continue this process until the end of the linked list.

Input Format

The first line consists of two integers M and N, separated by a space, representing the values to retain and delete.

The second line consists of a series of integers separated by spaces, representing the values of the linked list nodes.

The last input is -1 which will terminate the inputs.

vtu29414

Output Format

The output prints the linked list after applying the process of retaining every M node and deleting the next N nodes represented as a series of integers separated by "->".

Sample Test Case

Input: 2 2
1 2 3 4 5 6 7 8 -1

Output: 1->2->5->6

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

// insert at end
struct Node* insertEnd(struct Node* head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;

   if (head == NULL) return newNode;

   struct Node* temp = head;
   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
   return head;
}

// retain M delete N
struct Node* retainDelete(struct Node* head, int M, int N) {
   struct Node* curr = head;
   struct Node* temp;
```

```
    while (curr != NULL) {

        // skip M-1 nodes
        for (int i = 1; i < M && curr != NULL; i++) {
            curr = curr->next;
        }

        if (curr == NULL) break;

        // start deletion from next node
        temp = curr->next;

        // delete N nodes
        for (int i = 1; i <= N && temp != NULL; i++) {
            struct Node* del = temp;
            temp = temp->next;
            free(del);
        }

        // connect retained part
        curr->next = temp;

        // move to next retained node
        curr = temp;
    }

    return head;
}

// print list
void printList(struct Node* head) {
    struct Node* temp = head;

    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL)
            printf("->");
        temp = temp->next;
    }
}

int main() {
```

```
    int M, N, val;

    scanf("%d %d", &M, &N);

    struct Node* head = NULL;

    while (1) {
        scanf("%d", &val);
        if (val == -1) break;
        head = insertEnd(head, val);
    }

    head = retainDelete(head, M, N);

    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Imagine you are working on a text processing tool and need to implement a feature that allows users to insert characters at a specific position.

Implement a program that takes user inputs to create a singly linked list of characters and inserts a new character after a given index in the list.

Input Format

The first line of input consists of an integer N, representing the number of characters in the linked list.

The second line consists of a sequence of N characters, representing the linked list.

The third line consists of an integer index, representing the index(0-based) after which the new character node needs to be inserted.

The fourth line consists of a character value representing the character to be

inserted after the given index.

Output Format

If the provided index is out of bounds (larger than the list size):

1. The first line prints "Invalid index".
2. The second line prints "Updated list: " followed by the unchanged linked list values.

Otherwise, prints "Updated list: " followed by the updated space separated linked list after inserting the new character after the given index.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
a b c d e
2
X
Output: Updated list: a b c X d e

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   char data;
   struct Node* next;
};

// insert at end
struct Node* insertEnd(struct Node* head, char val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
```

```
    if (head == NULL) return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}

// print list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%c ", temp->data);
        temp = temp->next;
    }
}

// insert after index
int insertAfterIndex(struct Node* head, int index, char val) {
    struct Node* temp = head;
    int i = 0;

    while (temp != NULL && i < index) {
        temp = temp->next;
        i++;
    }

    if (temp == NULL) return 0; // invalid index

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;

    newNode->next = temp->next;
    temp->next = newNode;

    return 1;
}

int main() {
    int n, index;
```

```
    char val;

    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        char ch;
        scanf(" %c", &ch);
        head = insertEnd(head, ch);
    }

    scanf("%d", &index);
    scanf(" %c", &val);

    int ok = insertAfterIndex(head, index, val);

    if (!ok) {
        printf("Invalid index ");
    }

    printf("Updated list: ");
    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Imagine Anu is tasked with finding the middle element of a doubly linked list. Given a doubly linked list where each node contains an integer value and is inserted at the end, implement a program to find the middle element of the list. If the number of nodes is even, return the middle element pair.

Input Format

The first line of input consists of an integer N, representing the number of nodes in the doubly linked list.

The second line consists of N space-separated integers, representing the values of the nodes in the doubly linked list.

Output Format

The first line of output prints the space-separated elements of the doubly linked list.

The second line prints the middle element(s) of the doubly linked list, depending on whether the number of nodes is odd or even.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50

Output: 10 20 30 40 50
30

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

// insert at end
struct Node* insertEnd(struct Node* head, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->prev = newNode->next = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
```

```
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// print list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

// find middle
void printMiddle(struct Node* head) {
    struct Node* slow = head;
    struct Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // if even length, fast will not be NULL
    if (fast == NULL) {
        // even: need previous of slow as well
        printf("%d %d ", slow->prev->data, slow->data);
    } else {
        // odd: single middle
        printf("%d ", slow->data);
    }
}

int main() {
    int n, val;
    scanf("%d", &n);

    struct Node* head = NULL;
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertEnd(head, val);
    }

    printList(head);

    printf(" ");

    printMiddle(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Pranav wants to clockwise rotate a doubly linked list by a specified number of positions. He needs your help to implement a program to achieve this. Given a doubly linked list and an integer representing the number of positions to rotate, write a program to rotate the list clockwise.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated linked list elements.

The third line consists of an integer k, representing the number of positions to rotate the list.

Output Format

The output displays the elements of the doubly linked list after rotating it by k positions.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
1

Output: 5 1 2 3 4

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

// insert at end
struct Node* insertEnd(struct Node* head, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->prev = newNode->next = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// get length
int length(struct Node* head) {
    int len = 0;
    while (head != NULL) {
        len++;
```

```
        head = head->next;
    }
    return len;
}

// rotate clockwise by k
struct Node* rotate(struct Node* head, int k) {
    if (head == NULL || k == 0)
        return head;

    int n = length(head);

    // connect last to first
    struct Node* last = head;
    while (last->next != NULL)
        last = last->next;

    last->next = head;
    head->prev = last;

    // move to new tail (n-k steps)
    int steps = n - k;
    struct Node* newTail = head;

    for (int i = 1; i < steps; i++)
        newTail = newTail->next;

    struct Node* newHead = newTail->next;

    // break circle
    newTail->next = NULL;
    newHead->prev = NULL;

    return newHead;
}

// print list
void printList(struct Node* head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
```

```
}

int main() {
    int n, val, k;
    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertEnd(head, val);
    }

    scanf("%d", &k);

    head = rotate(head, k);

    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 5_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Arun is tasked with implementing a program to reverse a singly linked list. The program should populate the list elements by inserting them at the beginning, traverse the list and reverse the order of its elements.

Since Arun is new to programming, you have to guide him through the task.

Input Format

The first line of input consists of an integer N, representing the linked list size.

The second line consists of N space-separated list elements.

Output Format

The first line of output prints the list elements after inserting them at the beginning.

The second line prints the reversed linked list.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
23 85 47 62 31

Output: 31 62 47 85 23
23 85 47 62 31

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

// insert at beginning
struct Node* insertFront(struct Node* head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = head;
   return newNode;
}

// reverse linked list
struct Node* reverse(struct Node* head) {
   struct Node *prev = NULL, *curr = head, *next = NULL;

   while (curr != NULL) {
      next = curr->next;
      curr->next = prev;
      prev = curr;
      curr = next;
   }
```

```
    return prev;
}

// print list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main() {
    int n, val;
    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertFront(head, val);
    }

    // after inserting at beginning
    printList(head);

    printf(" ");

    // reverse list
    struct Node* revHead = reverse(head);

    // print reversed list
    printList(revHead);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

In a computer science course project, students are required to write a program to insert values at the end of a singly linked list and remove the last node from the list. The program should traverse the list and delete the last node, adjusting pointers accordingly.

Assist students in the task.

Input Format

The first line of input consists of an integer N, representing the list size.

The second line consists of N space-separated integers, representing the list elements.

Output Format

The output prints space-separated integers, representing the list after the deletion of the last element.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8
54 25 14 38 55 73 95 82
Output: 54 25 14 38 55 73 95

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

// insert at end
struct Node* insertEnd(struct Node* head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = val;
    newNode->next = NULL;

    if (head == NULL) {
        return newNode;
    }

    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    return head;
}

// delete last node
struct Node* deleteLast(struct Node* head) {
    if (head == NULL || head->next == NULL) {
        return NULL;
    }

    struct Node* temp = head;

    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    free(temp->next);
    temp->next = NULL;

    return head;
}

// print list
void printList(struct Node* head) {
    struct Node* temp = head;

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
```

```
}

int main() {
    int n, val;
    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertEnd(head, val);
    }

    head = deleteLast(head);

    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Dev is tasked with creating a program that efficiently finds the middle element of a linked list. The program should take user input to populate the linked list by inserting each element into the front of the list and then determining the middle element.

Assist Dev, as he needs to ensure that the middle element is accurately identified from the constructed singly linked list:

If it's an odd-length linked list, return the middle element.If it's an even-length linked list, return the second middle element of the two elements.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated integers, representing the elements of the list.

Output Format

The first line of output displays the linked list after inserting elements at the front.

The second line displays "Middle Element: " followed by the middle element of the linked list.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50
Output: 50 40 30 20 10
Middle Element: 30

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

// insert at front
struct Node* insertFront(struct Node* head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = head;
   return newNode;
}

// print list
void printList(struct Node* head) {
   struct Node* temp = head;
   while (temp != NULL) {
      printf("%d ", temp->data);
```

```
        temp = temp->next;
    }
}

// find middle (second middle for even length)
int findMiddle(struct Node* head) {
    struct Node* slow = head;
    struct Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow->data;
}

int main() {
    int n, val;
    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertFront(head, val);
    }

    printList(head);

    printf(" Middle Element: %d", findMiddle(head));

    return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

Saran is tasked with implementing a doubly linked list in data structures, allowing users to append elements to the end of the list, and remove the

last element. He needs to ensure efficient memory management in the implementation.

Assist him with a suitable program.

Input Format

The first line of input consists of an integer n, representing the size of the list.

The second line consists of n space-separated integers, representing the elements of the list.

Output Format

The output displays integers, representing the list after removing the last element, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
48 22 5 62 87

Output: 48 22 5 62

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

// insert at end
struct Node* insertEnd(struct Node* head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->prev = NULL;
```

```
    newNode->next = NULL;

    if (head == NULL) {
        return newNode;
    }

    struct Node* temp = head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// delete last node
struct Node* deleteLast(struct Node* head) {
    if (head == NULL || head->next == NULL) {
        return NULL;
    }

    struct Node* temp = head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->prev->next = NULL;
    free(temp);

    return head;
}

// print list
void printList(struct Node* head) {
    struct Node* temp = head;

    while (temp != NULL) {
        printf("%d ", temp->data);
```

```
        temp = temp->next;
    }
}

int main() {
    int n, val;
    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertEnd(head, val);
    }

    head = deleteLast(head);

    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement:

Ravi is building a simple application to manage a doubly linked list of integers. He wants to insert new integers into the list and delete a node at a specific position. After deleting it, he needs to display the updated list.

Write a program to ensure that the task is done by performing insertion, deletion at a given position, and then printing the modified list.

Input Format

The first line contains an integer n, representing the number of elements to be inserted into the list.

The second line contains n space-separated integers, representing the values to insert.

The third line contains an integer pos, representing the position of the node to be deleted from the list.

Output Format

If the given position is valid, print "Node at position X with value Y deleted successfully".

If the position is invalid, print "Position X not found in the list."

In both cases, print

"Data present in the list:"

followed by the updated list elements on the next line, space separated."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50
4
Output: Node at position 4 with value 40 deleted successfully.
Data present in the list:
10 20 30 50

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
```

```
};

// insert at end
struct Node* insertEnd(struct Node* head, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->prev = newNode->next = NULL;

    if (head == NULL) return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// delete at position (1-based)
struct Node* deleteAtPos(struct Node* head, int pos, int *deletedVal, int *found)
{
    if (head == NULL) return head;

    struct Node* temp = head;

    int count = 1;

    while (temp != NULL && count < pos) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL) {
        *found = 0;
        return head;
    }

    *found = 1;
    *deletedVal = temp->data;
```

```
    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else
        head = temp->next; // deleting head

    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    free(temp);

    return head;
}

// print list
void printList(struct Node* head) {
    struct Node* temp = head;

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main() {
    int n, val, pos;

    scanf("%d", &n);

    struct Node* head = NULL;

    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertEnd(head, val);
    }

    scanf("%d", &pos);

    int deletedVal = -1;
    int found = 0;

    head = deleteAtPos(head, pos, &deletedVal, &found);
```

```
    if (found) {
        printf("Node at position %d with value %d deleted successfully. ", pos,
deletedVal);
    } else {
        printf("Position %d not found in the list. ", pos);
    }

    printf("Data present in the list: ");
    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 6

Section 1 : MCQ

1. After adding a new node at the end of a linked list, what does the current last node become?

Answer

The second last node

Status : Correct ***Marks : 1/1***

2. When inserting a new element at the beginning of a linked list, what happens to the current head of the list?

Answer

It becomes the second element

Status : Correct ***Marks : 1/1***

vtu29414

3. Which of the following are true about a singly linked list?

Answer

Each node has a reference to both the previous and next node.

Status : Wrong ***Marks : 0/1***

4. When inserting a new node into a singly linked list, which of the following are the possible cases for insertion?

Answer

 Inserting a new node at the middle of the list (random location)

Status : Wrong ***Marks : 0/1***

5. Given the linked list: 5 -> 10 -> 15 -> 20 -> 25 -> NULL. What will be the output of traversing the list and printing each node's data?

Answer

25 20 15 10 5

Status : Wrong ***Marks : 0/1***

6. Which of the following is the process of finding information in a linked list?

Answer

Start from the head of the list and traverse sequentially until finding (or not finding) the node

Status : Correct ***Marks : 1/1***

7. Which of the following actions needs to be taken when the last item in a linked list is deleted?

Answer

vtu29414

Change the reference of the second last item to NULL reference

Status : Correct ***Marks : 1/1***

8. Which pointer helps in traversing a doubly linked list in reverse order?

Answer

head

Status : Wrong ***Marks : 0/1***

9. Which of the following is true about the last node in a doubly linked list?

Answer

Both prev and next pointers are NULL

Status : Wrong ***Marks : 0/1***

10. What will be the effect of setting the prev pointer of a node to NULL in a doubly linked list?

Answer

The node will become the new head

Status : Correct ***Marks : 1/1***

11. How do you delete a node from the middle of a doubly linked list?

Answer

Update the next pointer of the previous node

Status : Wrong ***Marks : 0/1***

12. What happens if we insert a node at the beginning of a doubly linked list?

Answer

vtu29414

The previous pointer of the head node is not updated

Status : Wrong ***Marks : 0/1***

13. How many pointers does a node in a doubly linked list have?

Answer

2

Status : Correct ***Marks : 1/1***

14. In deletion from the end of a doubly linked list, which pointer needs to be updated?

Answer

head->prev = NULL

Status : Wrong ***Marks : 0/1***

15. Which of the following is true about a Doubly Linked List?

Answer

First node's prev pointer points to the last node

Status : Wrong ***Marks : 0/1***

vtu29414

Vel Tech Group of Institutions

Name: ARIKATLA AJAY KUMAR Ajaykumar
Email: vtu29414@veltech.edu.in
Roll no: vtu29414
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L6

VTU_Week 4_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

A financial analytics company is developing a tool to analyze stock price trends. The tool needs to identify the Next Greater Price for each stock price in a given list of daily prices. For a given stock price, the Next Greater Price is defined as the first price appearing after it in the list that is greater. If no such price exists, the result should be -1.

Note: The Next greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider the next greater element as -1.

Examples:

vtu29414

Input Format

The first line of input consists of an integer n, representing the size of the array.

The second line consists of n space-separated stack of integers.

Output Format

The output prints "Next Greater Elements: " followed by an array of integers, representing the next greater elements in the stack.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
4 5 2 25

Output: Next Greater Elements: 5 25 25 -1

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Next Greater Elements: ");

    for (int i = 0; i < n; i++) {
        int nextGreater = -1;

        for (int j = i + 1; j < n; j++) {
            if (arr[j] > arr[i]) {
                nextGreater = arr[j];
```

```
            break;
        }
    }

    printf("%d ", nextGreater);
  }

  return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement:

Riya is working on a text-cleaning utility where certain unwanted patterns must be removed from user input. She uses a stack data structure to process the string character by character.

Given a string containing alphabets, digits, and special characters, write a program to ensure that the task is done by removing every digit along with the closest non-digit character immediately to its left. The remaining characters should be preserved in their original order and printed as the final cleaned string.

Example:

Input:

s = "cb34"

Output:

""

Explanation:

Start with an empty stack.

Push c stack: ['c']

Push b stack: ['c','b']

Encounter 3 pop 'b' stack: ['c']

Encounter 4 pop 'c' stack: []

Result: empty string

Input Format

The input consists of a string s without spaces, consisting of letters, digits, and special characters.

Output Format

The output prints the resulting string after performing the required removals.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abc

Output: abc

Answer

```
// You are using GCC
#include <stdio.h>
#include <ctype.h>

int main() {
    char s[105];
    scanf("%s", s);

    char stack[105];
    int top = -1;

    for (int i = 0; s[i] != '\0'; i++) {
        if (isdigit(s[i])) {
            // remove closest non-digit to the left
            if (top >= 0) {
                top--;
            }
        } else {
            // push character
            stack[++top] = s[i];
```

```
        }
    }

    // print result
    for (int i = 0; i <= top; i++) {
        printf("%c", stack[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

3. Problem Statement

Kavya is developing a real-time analytics system that processes a continuous stream of numerical data. To efficiently track trends, she uses a Queue data structure to maintain incoming values and compute statistics incrementally.

Given a sequence of integers arriving one by one, write a program to ensure that the task is done by calculating and printing the moving average of all elements received so far after each new input. The average should be displayed up to two decimal places.

Example

Input

3

1 2 3

Output

1.00 1.50 2.00

Explanation

The moving average of the first element is 1.00.

The next elements are 1 and 2, the moving average is (1 + 2) / 2 = 1.50.

The next elements are 1, 2, and 3, the moving average is (1 + 2 + 3) / 3 = 2.00.

Input Format

The first line contains an integer n, representing the number of elements in the data stream.

The second line contains n space-separated integers representing the incoming values.

Output Format

The output prints n floating-point numbers, each representing the moving average after processing the corresponding element.

Each value should be printed with two decimal places, separated by spaces.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
1 2 3
Output: 1.00 1.50 2.00

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[15];
    double sum = 0.0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        sum += arr[i];
```

```
    double avg = sum / (i + 1);
    printf("%.2f ", avg);
  }

  return 0;
}
```

Status : Correct ***Marks : 10/10***

4. Problem Statement

In a small business, Jane is tasked with organizing customer feedback scores for a new product. Each customer has rated the product, and Jane needs to arrange these ratings in a specific order to analyze the feedback more effectively.

Specifically, she wants to rearrange the ratings such that the most frequent ratings appear first. If two ratings have the same frequency, the smaller rating should come first. To accomplish this, assist Jane with a program to process a queue of customer ratings and rearrange them based on their frequencies.

Input Format

The first line contains an integer n, which represents the number of customer ratings.

The second line contains n integers, each representing a customer rating.

Output Format

The output displays the rearranged ratings in the order of their frequency (most frequent first).

If frequencies are the same, the smaller rating should come first.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7
2 3 2 3 2 3 2
Output: 2 2 2 2 3 3 3

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];
    int freq[100] = {0};

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        freq[arr[i]]++;
    }

    // sort unique values based on frequency and value
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int a = arr[i], b = arr[j];

            if (freq[a] < freq[b] ||
               (freq[a] == freq[b] && a > b)) {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    // print result
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***

vtu29414